Machine, Data, and Learning (MDL)

Comprehensive Exam Study Guide

Spring 2026 — IIIT Hyderabad

Synthesized from Lectures, Slides, and Class Notes

Compiled by Harith Y.

April 26, 2026

Contents

Chapter 1

Introduction to Artificial Intelligence and Machine Learning

Based on Lecture of January 3, 2026 — Lecture Slides: lec1.pdf

1.1 Course Overview

The MDL (Machine, Data, and Learning) course is organized into 5 modules covering AI and ML topics. The evaluation structure is:

- **Assignments:** 4 total — first assignment worth 4%, remaining three worth 7% each (total 25%)
- **Quizzes:** 2 quizzes, each worth 7.5% (total 15%)
- **Midterm Exam** and **Final Exam** (remaining 60%)

Reference Texts:

- *Artificial Intelligence: A Modern Approach*, 4th Edition — Stuart Russell and Peter Norvig
- Various Python ML books (freely available online)

Harith's Insight / Class Context

The instructor emphasized a strict no-digital-device policy in class. Handwritten notes are recommended for better retention. This study guide incorporates those handwritten class notes throughout.

1.2 What is Artificial Intelligence?

Definition

Artificial Intelligence (AI) is the science and engineering of making machines that can think and act like humans, or more broadly, that can think and act rationally.

AI can be understood through two dimensions:

1. **Human-like vs. Rational**: Does the system aim to mimic human behavior, or does it aim for ideal (rational) behavior?
2. **Thinking vs. Acting**: Does the system focus on internal reasoning processes, or on external behavior and actions?

This gives rise to **four quadrants** of AI:

	Humanly	**Rationally**
Think	Cognitive Science	Logic / Formal Methods
Act	Turing Test	Rational Agents

1.2.1 The Turing Test

The **Turing Test**, proposed by Alan Turing in 1950, tests whether a machine can exhibit intelligent behavior indistinguishable from that of a human. A human evaluator interacts with a machine and a human through text; if the evaluator cannot reliably distinguish the machine from the human, the machine is said to pass the test.

Key Concept

The Turing Test evaluates *acting humanly*. It does not test whether a machine truly "thinks" — only whether its behavior is indistinguishable from human behavior.

1.3 Machines: Simple and Complex

Definition

A **machine** is a device that makes life easier using mechanical power.

Definition

Intelligence is the ability to understand, learn, and think.

- **Simple machines:** Lever, pulley, inclined plane — perform basic mechanical functions.
- **Complex machines:** Computers, robots — can perform sophisticated computations and learning.

1.4 What is Machine Learning?

Definition

Machine Learning (ML) is a subset of AI that enables machines to learn from data and make predictions or decisions without being explicitly programmed for every scenario. It focuses on the construction of algorithms that learn from and make predictions based on data.

ML operates on the principle that systems can learn patterns from data, identify structures, and make inferences with minimal human intervention.

1.4.1 Types of Machine Learning

1. **Supervised Learning:** The algorithm is trained on *labeled* data. Each training example consists of an input and a known correct output (label).
   - **Classification:** Output is a discrete category (e.g., spam vs. not spam).
   - **Regression:** Output is a continuous value (e.g., house price prediction).
2. **Unsupervised Learning:** The algorithm is trained on *unlabeled* data to discover hidden patterns, groupings, or structures.
   - **Clustering:** Grouping similar data points together.
   - Examples: Market segmentation, biological taxonomy, earthquake study patterns, web page clustering.
3. **Reinforcement Learning:** An agent learns by interacting with an environment, receiving rewards or penalties for actions, and optimizing its strategy to maximize cumulative reward.
   - The agent-environment interaction loop: State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ New State.

Key Concept

The fundamental distinction: In **supervised learning**, we have ground truth labels. In **unsupervised learning**, we discover structure without labels. In **reinforcement learning**, we learn from delayed reward signals.

1.4.2 The Perceptron Model

The lecture introduced the **perceptron** as the simplest neural network unit:

- Takes multiple inputs $x_1, x_2, \ldots, x_n$ with weights $w_1, w_2, \ldots, w_n$.
- Computes a weighted sum: $z = \sum_{i=1}^{n} w_i x_i + b$ (where b is the bias).
- Applies an activation function to produce the output.

Example: Classifying a cricket ball vs. a tennis ball using features like diameter and weight — a simple binary classification problem solvable by a perceptron.

1.5 Ethics and Fairness in AI

Key Concept

AI systems can perpetuate and amplify existing biases present in training data. Ensuring **fairness**, **transparency**, and **accountability** in AI is a critical ongoing challenge.

- AI applications in healthcare, finance, and criminal justice raise ethical concerns.

- Models trained on biased data can produce discriminatory outcomes.
- **Responsible AI** involves building systems that are fair, interpretable, and accountable.

1.6 AI Applications

AI is deployed across numerous domains:

- **Healthcare:** Disease diagnosis, drug discovery, medical imaging.
- **Finance:** Fraud detection, algorithmic trading, risk assessment.
- **Daily Life:** Search engines, recommendation systems, voice assistants.
- **Transportation:** Autonomous vehicles, route optimization.
- **Science:** Protein folding (AlphaFold), climate modeling.

1.7 Neural Network Scale

The lecture noted the dramatic growth of neural network complexity:

- Early networks had millions of parameters.
- Modern large language models have grown from billions to **trillions of parameters**.
- This growth is enabled by increasing computational power and data availability.

1.8 Practice Questions with Solutions

1. **Define Artificial Intelligence. What are the four quadrants of AI research?**

   Solution: AI is the science and engineering of making machines that can think and act intelligently. The four quadrants arise from two dimensions — *Human-like vs. Rational* and *Thinking vs. Acting*:

   - **Thinking Humanly:** Cognitive science approach — model human thought processes.
   - **Acting Humanly:** Turing Test approach — produce behavior indistinguishable from humans.
   - **Thinking Rationally:** Logic / formal methods approach — use logical inference to reach correct conclusions.
   - **Acting Rationally:** Rational agent approach — take actions that maximize goal achievement.

2. **Explain the difference between supervised, unsupervised, and reinforcement learning with examples.**

   Solution:

   - **Supervised learning:** Trained on labeled input-output pairs. The model learns a mapping $f : X \to Y$. *Example:* Email spam detection — each email is labeled spam/not-spam.
   - **Unsupervised learning:** Trained on unlabeled data; discovers hidden structure. *Example:* Customer segmentation — group customers by purchasing behavior without predefined categories.
   - **Reinforcement learning:** An agent learns by interacting with an environment, receiving rewards/penalties. *Example:* AlphaGo learning to play Go through self-play, receiving reward for winning.

   Key distinction: supervised has ground truth labels, unsupervised discovers structure, RL learns from delayed reward signals.

3. **What is the Turing Test? What does it evaluate?**

   Solution: The Turing Test (proposed by Alan Turing, 1950) evaluates whether a machine can exhibit intelligent behavior indistinguishable from a human. A human evaluator communicates via text with both a machine and a human. If the evaluator cannot reliably distinguish them, the machine passes. It evaluates *acting humanly* — behavioral equivalence, not whether the machine truly "thinks."

4. **Describe the perceptron model. How does it perform binary classification?**

   Solution: The perceptron takes inputs $x_1, \ldots, x_n$ with weights $w_1, \ldots, w_n$ and a bias b. It computes the weighted sum $z = \sum_{i=1}^{n} w_i x_i + b$, then applies an activation function (e.g., step function): output $= 1$ if $z \geq 0$, else 0. This creates a linear decision boundary $\mathbf{w} \cdot \mathbf{x} + b = 0$ that separates two classes. *Example:* Classifying cricket ball vs. tennis ball using diameter and weight as features.

5. **Why is fairness important in AI? Give an example of how bias can arise in ML systems.**

   Solution: AI systems make consequential decisions (hiring, lending, sentencing). If training data reflects historical biases, the model learns and amplifies them. *Example:* A hiring model trained on historical data where most hires were male may learn to penalize female applicants, perpetuating gender discrimination — even though gender was not an explicit input feature (proxy features like name or university can encode it).

Chapter 2

Logic and Propositional Reasoning

Based on Lectures of January 7 and January 10, 2026 — Lecture Slides: lec2.pdf, lec3.pdf

2.1 Knowledge Representation

Definition

Knowledge Representation is the process of expressing knowledge explicitly in a computer-tractable form so that it can be used for reasoning and inference.

- A **knowledge base (KB)** is a set of sentences expressed in a knowledge representation language.
- **Knowledge-based agents** use a knowledge base to make decisions and take actions.

2.2 Formal vs. Informal Languages

Key Concept

Natural (informal) languages exhibit **ambiguity**. Formal languages provide **precision** and eliminate implicit assumptions, reducing human errors in reasoning.

- **Informal languages:** Natural language (English, Hindi, etc.) — ambiguous, context-dependent.
- **Formal languages:** Mathematical logic, programming languages — precise, unambiguous.
- Formal languages are essential for AI because machines require unambiguous instructions.

2.3 Logic: Foundations

Definition

Logic is a branch of mathematics and philosophy that deals with reasoning, argumentation, and inference. It provides rules for deriving valid conclusions from given premises.

Types of logic, organized by **order**:

- **Propositional Logic (Zeroth-Order Logic):** Deals with propositions that are either true or false. No quantifiers.
- **First-Order Logic (Predicate Logic):** Extends propositional logic with quantifiers ($\forall$, $\exists$) and predicates over objects.
- **Second-Order Logic:** Quantifies over sets and relations, not just individuals.
- **Higher-Order Logic:** Extends second-order logic with quantification over higher-order entities.

Harith's Insight / Class Context

The instructor emphasized that mathematical logic forms the foundation of AI. Symbolic logic is a subset of formal logic, and propositional logic serves as the entry point for knowledge representation and reasoning.

2.4 Propositional Logic

2.4.1 Propositions

Definition

A **proposition** is a declarative statement that is either **true** or **false**, but not both.

- **Atomic propositions:** Basic facts or conditions (e.g., "It is raining").
- **Compound propositions:** Formed by combining atomic propositions using logical connectives.

Not propositions: Questions, commands, exclamations (e.g., "Close the door!" is not a proposition).

2.4.2 Logical Connectives

The five fundamental logical connectives:

Connective	Symbol	Name	Meaning
NOT	$\neg$	Negation	"It is not the case that P"
AND	$\wedge$	Conjunction	"P and Q"
OR	$\vee$	Disjunction	"P or Q" (inclusive)
IMPLIES	$\rightarrow$	Implication	"If P then Q"
IFF	$\leftrightarrow$	Biconditional	"P if and only if Q"

2.4.3 Truth Tables

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \to Q$	$P \leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

Exam Tip

The implication $P \to Q$ is **only false** when P is true and Q is false. When P is false, the implication is **vacuously true** regardless of Q. This is the most commonly tested connective.

2.4.4 Key Terminology

Definition

Tautology: A formula that is **true under all possible truth value assignments**. Example: $P \vee \neg P$.

Definition

Satisfiable: A formula is satisfiable if there **exists at least one assignment** of truth values that makes it true.

Definition

Unsatisfiable (Contradiction / Fallacy): A formula that is **false under all possible truth value assignments**. Example: $P \wedge \neg P$.

Definition

Valid: A formula is valid if it is true under **all possible assignments**. A valid formula is a tautology.

Key Concept

Every tautology is satisfiable, but not every satisfiable formula is a tautology. An unsatisfiable formula is *not* satisfiable.

2.5 Formal Proofs and Inference Rules

(From Lecture of January 10, 2026 — lec3.pdf)

2.5.1 What is a Formal Proof?

Definition

A **formal proof** is a sequence of steps where each step is either:

1. An **axiom** or **premise** (given as true),
2. A **formula deduced** from previous steps using a **rule of inference**.

We write $\alpha \Rightarrow \beta$ to mean that β can be derived from α.

2.5.2 Properties of Logical Systems

- **Soundness:** If a formula can be derived (proved), then it is true. (Proof $\Rightarrow$ Truth)
- **Completeness:** If a formula is true, then it can be derived. (Truth $\Rightarrow$ Proof)
- **Decidability:** There exists an algorithm that can determine, for any formula, whether it is a theorem.

Key Concept

A logical system is ideal when it is both **sound** and **complete**. Propositional logic is both sound and complete. First-order logic is sound and complete (Gödel's completeness theorem) but **not decidable** in general.

2.5.3 Standard Inference Rules

1. **Modus Ponens:** From P and $P \to Q$, infer Q.

$$\frac{P, \quad P \to Q}{Q} \tag{2.1}$$

2. **Modus Tollens:** From $\neg Q$ and $P \to Q$, infer $\neg P$.

$$\frac{\neg Q, \quad P \to Q}{\neg P} \tag{2.2}$$

3. **Hypothetical Syllogism:** From $P \to Q$ and $Q \to R$, infer $P \to R$.

$$\frac{P \to Q, \quad Q \to R}{P \to R} \tag{2.3}$$

4. **AND Elimination:** From $P \wedge Q$, infer P (or Q).

$$\frac{P \wedge Q}{P} \tag{2.4}$$

5. **AND Introduction:** From P and Q, infer $P \wedge Q$.

$$\frac{P, \quad Q}{P \wedge Q} \tag{2.5}$$

6. **OR Introduction:** From P, infer $P \vee Q$ (for any Q).

$$\frac{P}{P \vee Q} \tag{2.6}$$

7. **Double Negation Elimination:** From $\neg\neg P$, infer P.

$$\frac{\neg\neg P}{P} \tag{2.7}$$

8. **Unit Resolution:** From $P \vee Q$ and $\neg P$, infer Q.

$$\frac{P \vee Q, \quad \neg P}{Q} \tag{2.8}$$

9. **Disjunctive Syllogism:** Same as unit resolution (from $P \vee Q$ and $\neg P$, infer Q).

Harith's Insight / Class Context

From handwritten notes: A **fallacy** occurs when reasoning is incorrect — but note that the conclusion may still happen to be correct by coincidence. The key issue is that the *reasoning process* is invalid.

2.5.4 Normal Forms

Definition

A **literal** is a propositional variable or its negation (e.g., P or $\neg P$).

Definition

A **clause** is a disjunction of literals (e.g., $P \vee \neg Q \vee R$).

Definition

Conjunctive Normal Form (CNF): A conjunction of clauses.

$$\text{CNF: } (l_{11} \vee l_{12} \vee \cdots) \wedge (l_{21} \vee l_{22} \vee \cdots) \wedge \cdots$$

Definition

Disjunctive Normal Form (DNF): A disjunction of conjunctions of literals.

$$\text{DNF: } (l_{11} \wedge l_{12} \wedge \cdots) \vee (l_{21} \wedge l_{22} \wedge \cdots) \vee \cdots$$

Exam Tip

Converting to CNF is a standard exam question. Steps: (1) Eliminate $\leftrightarrow$ and $\rightarrow$, (2) Push $\neg$ inward using De Morgan's laws, (3) Distribute $\vee$ over $\wedge$.

Conversion to CNF — Step-by-Step:

1. Replace $P \leftrightarrow Q$ with $(P \rightarrow Q) \wedge (Q \rightarrow P)$.

2. Replace $P \to Q$ with $\neg P \vee Q$.
3. Apply De Morgan's Laws: $\neg(P \wedge Q) = \neg P \vee \neg Q$ and $\neg(P \vee Q) = \neg P \wedge \neg Q$.
4. Eliminate double negations: $\neg\neg P = P$.
5. Distribute $\vee$ over $\wedge$: $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$.

Harith's Insight / Class Context

Robin's Conjecture was proven by a computer using logic — demonstrating the power of automated theorem proving, a practical application of the formal proof systems discussed in class.

2.6 Practice Questions with Solutions

1. **Construct the truth table for $(P \to Q) \leftrightarrow (\neg Q \to \neg P)$. Is it a tautology?**

   Solution: Note that $\neg Q \to \neg P$ is the **contrapositive** of $P \to Q$, and a statement is always logically equivalent to its contrapositive.

P	Q	$P \to Q$	$\neg Q$	$\neg P$	$\neg Q \to \neg P$	Biconditional
T	T	T	F	F	T	**T**
T	F	F	T	F	F	**T**
F	T	T	F	T	T	**T**
F	F	T	T	T	T	**T**

   Yes, it is a tautology — the biconditional is true for all assignments.

2. **Convert $(P \to Q) \wedge (Q \to R)$ to CNF.**

   Solution:

   (a) Eliminate $\to$: $(\neg P \vee Q) \wedge (\neg Q \vee R)$

   (b) This is already in CNF (conjunction of clauses where each clause is a disjunction of literals).

   CNF: $(\neg P \vee Q) \wedge (\neg Q \vee R)$.

3. **Using Modus Ponens and Hypothetical Syllogism, prove that from P, $P \to Q$, and $Q \to R$, we can derive R.**

   Solution:

   (a) P (Premise)

   (b) $P \to Q$ (Premise)

   (c) Q (Modus Ponens on 1 and 2)

   (d) $Q \to R$ (Premise)

   (e) R (Modus Ponens on 3 and 4) ■

   Alternative (using Hypothetical Syllogism):

(a) $P \to Q$ and $Q \to R \Rightarrow P \to R$ (Hypothetical Syllogism)

(b) P and $P \to R \Rightarrow R$ (Modus Ponens) ■

4. **Determine whether $(P \vee Q) \wedge (\neg P \vee R) \wedge (\neg Q) \wedge (\neg R)$ is satisfiable.**

   Solution: From clause $(\neg Q)$: $Q = F$. From clause $(\neg R)$: $R = F$. Substituting into $(\neg P \vee R)$: $\neg P \vee F = \neg P$, so $P = F$. Substituting into $(P \vee Q)$: $F \vee F = F$. This clause is **false**. Since no assignment satisfies all clauses simultaneously, the formula is **unsatisfiable**.

5. **Explain the difference between soundness and completeness. Why are both important?**

   Solution:

   - **Soundness:** Every provable statement is true. (No false theorems can be derived.) This ensures *correctness* — the system never lies.
   - **Completeness:** Every true statement is provable. (Nothing true is missed.) This ensures *power* — the system can derive everything that holds.

   Both are needed: a sound but incomplete system is trustworthy but limited. A complete but unsound system can prove everything (including falsehoods), making it useless. Propositional logic is both sound and complete.

6. **What is a fallacy? Can a fallacious argument lead to a true conclusion? Explain.**

   Solution: A **fallacy** is an argument whose reasoning is logically invalid — the conclusion does not follow from the premises by valid inference rules. **Yes**, a fallacious argument can have a true conclusion by coincidence. Example: "All birds can fly. Penguins are birds. Therefore penguins can fly." The reasoning is valid but the premise is false. Conversely: "If it rains, the ground is wet. The ground is wet. Therefore it rained" — this is the fallacy of *affirming the consequent*, yet the conclusion might happen to be true (it may indeed have rained). The issue is that the *reasoning process* is invalid, not necessarily the conclusion.

Chapter 3

Machine Learning: Foundations and Breakthroughs

Based on Lectures of January 17 and January 21, 2026 — Lecture Slides: lec4.pdf

3.1 Machine Learning: Definition and Philosophy

Definition

Machine Learning (term coined circa 1960): The construction of algorithms that can learn from data to make predictions or decisions. ML enables computers to improve their performance on a task through experience (data), without being explicitly programmed.

Why Learn Machine Learning?

- Patterns in data are too complex for humans to specify manually.
- Dynamic events require adaptive systems.
- Exponential data growth demands automated analysis.
- Diverse data formats (text, images, audio, video) require flexible models.
- Data quality and preprocessing determine model success.

3.2 The Three Categories of Machine Learning (Detailed)

3.2.1 Supervised Learning

- Training data consists of input-output pairs (x_i, y_i).
- The algorithm learns a function $f : X \to Y$ that maps inputs to outputs.
- **Classification:** Y is discrete (categorical). Example: email $\to$ {spam, not spam}.
- **Regression:** Y is continuous. Example: house features $\to$ price.

- The correct output is called the **ground truth** or **label**.

3.2.2 Unsupervised Learning

- Training data consists of inputs x_i only — no labels.
- The algorithm discovers structure or patterns in the data.
- **Clustering:** Grouping similar data points (e.g., customer segmentation).
- **Applications:** Marketing, biology (taxonomy), earthquake studies, web page organization.

3.2.3 Reinforcement Learning

- An **agent** interacts with an **environment**.
- The agent takes **actions** based on the current **state**.
- The environment provides **rewards** (or penalties) and transitions to a new state.
- Goal: Learn a **policy** π that maximizes cumulative reward.
- Key concept: **Delayed reward** — the agent may not receive immediate feedback for its actions.

3.3 Landmark AI Systems

3.3.1 AlphaGo

- Developed by DeepMind; defeated world champion Lee Sedol in 2016.
- Used **Monte Carlo Tree Search (MCTS)** combined with deep neural networks.
- MCTS is a randomized search technique that builds a search tree using random sampling.
- **Self-play:** AlphaGo improved by playing millions of games against itself.
- Demonstrated that ML can master games with enormous search spaces (10^{170} possible Go positions).

3.3.2 AlphaZero and MuZero

- **AlphaZero:** Generalized AlphaGo's approach to chess, shogi, and Go. Learned purely through self-play without human game data.
- **MuZero:** Further generalization — can master games **without even knowing the rules**. Learns the rules and optimal play simultaneously.

3.3.3 AlphaFold

- Addresses the **protein folding problem**: predicting a protein's 3D structure from its amino acid sequence.
- This was a 50-year grand challenge in biology.
- **Levinthal's Paradox:** A protein with 100 amino acids has 10^{300} possible configurations, yet proteins fold in milliseconds. This suggests that folding follows a specific pathway, not random search.
- AlphaFold achieved breakthrough accuracy in the CASP competition.

Key Concept

Levinthal's Paradox: It would take longer than the age of the universe to enumerate all possible protein conformations, yet proteins fold in microseconds to milliseconds. This implies that protein folding follows a directed pathway, which ML can learn.

3.3.4 Nobel Prizes and Turing Award (2024)

- **Nobel Prize in Chemistry 2024:** Awarded for AI contributions to protein design and structure prediction.
- **Nobel Prize in Physics 2024:** Awarded for foundational work on machine learning with neural networks.
- **Turing Award 2024:** Awarded for contributions to reinforcement learning.

3.4 Search Techniques

- **Breadth-First Search (BFS):** Explores all nodes at the current depth before moving deeper. Guarantees shortest path in unweighted graphs.
- **Depth-First Search (DFS):** Explores as deep as possible along each branch before backtracking. Memory-efficient but may not find shortest path.
- **A* Search:** Uses a heuristic $h(n)$ to estimate cost-to-goal. Combines with actual cost $g(n)$: $f(n) = g(n) + h(n)$.
- **Monte Carlo Tree Search (MCTS):** Randomized technique used in game-playing AI. Builds a search tree via simulation and backpropagation of results.

Harith's Insight / Class Context

From handwritten notes: Search can be categorized as **randomized** (MCTS) vs. **non-randomized** (BFS, DFS, A*). Randomized techniques are particularly effective when the search space is too large for exhaustive exploration.

3.5 Stanford AI Index 2025

- AI performance on demanding benchmarks is approaching or exceeding human baseline.
- Industry (not academia) dominates AI development in terms of compute and model scale.
- Computing power used for training state-of-the-art models is growing exponentially.

3.6 Applications of Machine Learning

- Search engines and information retrieval
- Recommendation systems (Netflix, Amazon)
- Fraud detection in financial transactions
- Disease diagnosis and medical imaging
- Election prediction and social science
- Image processing and computer vision
- Speech recognition and translation

3.7 Practice Questions with Solutions

1. **Explain the three categories of machine learning with real-world examples for each.**

   Solution: (1) **Supervised:** learns from labeled data. Example: predicting house prices from features (regression), classifying images as cat/dog (classification). (2) **Unsupervised:** discovers patterns in unlabeled data. Example: clustering customers by purchase behavior, topic modeling in documents. (3) **Reinforcement:** agent learns via trial-and-error with rewards. Example: a robot learning to walk, AlphaGo learning Go via self-play.

2. **What is the significance of AlphaGo defeating Lee Sedol? How does Monte Carlo Tree Search work?**

   Solution: Go has $\sim 10^{170}$ possible board positions, making exhaustive search impossible. AlphaGo's 2016 victory showed that ML can master domains previously thought to require human intuition. **MCTS** works by: (1) *Selection:* traverse the tree using a policy to pick promising nodes; (2) *Expansion:* add a new child node; (3) *Simulation (rollout):* play random games from the new node to estimate its value; (4) *Backpropagation:* update the statistics of all ancestor nodes. AlphaGo combined MCTS with deep neural networks that guided both selection and value estimation.

3. **Explain Levinthal's Paradox. How does AlphaFold address the protein folding problem?**

   Solution: A protein with 100 amino acids has $\sim 10^{300}$ possible 3D conformations. Exhaustively searching these would take longer than the age of the universe. Yet proteins fold in microseconds to milliseconds — this is **Levinthal's Paradox**. It implies folding follows a directed pathway, not random search. **AlphaFold** uses deep learning to predict a protein's 3D structure from its amino acid sequence, learning the folding "rules" from known structures. It achieved breakthrough accuracy in the CASP competition, effectively solving the 50-year grand challenge.

4. **Compare BFS, DFS, and A* in terms of completeness, optimality, and space complexity.**

   Solution:

	Complete?	**Optimal?**	**Space**
BFS	Yes (finite)	Yes (uniform cost)	$O(b^d)$
DFS	No (can loop)	No	$O(bd)$
A*	Yes (admissible h)	Yes (admissible h)	$O(b^d)$

   where b = branching factor, d = depth of solution. A* uses $f(n) = g(n) + h(n)$ and is optimal when h is admissible (never overestimates).

5. **What role does self-play have in reinforcement learning systems like AlphaZero?**

   Solution: Self-play allows the agent to generate its own training data by playing against itself. Each game produces states, actions, and outcomes that serve as training examples. Over millions of games, the agent explores diverse strategies, discovers weaknesses, and improves iteratively. Crucially, self-play removes the need for human expert data — AlphaZero learned chess from scratch and surpassed all human knowledge purely through self-play.

6. **Why is the distinction between supervised and unsupervised learning fundamental in ML?**

   Solution: The distinction determines the entire learning paradigm. With labels (supervised), we optimize a known objective (minimize prediction error on known outputs). Without labels (unsupervised), we must define what "structure" means (e.g., cluster compactness). This affects algorithm design, evaluation metrics (accuracy vs. silhouette score), and applicable problem types. Many real-world problems lack labels, making unsupervised methods essential.

7. **Describe the agent-environment interaction loop in RL. What is a policy?**

   Solution: The loop: (1) Agent observes current **state** s_t. (2) Agent selects an **action** a_t based on its policy. (3) Environment transitions to new state s_{t+1} and returns a **reward** r_t. (4) Repeat. A **policy** π is a mapping from states to actions: $\pi : S \to A$ (deterministic) or $\pi(a \mid s)$ giving a probability distribution over actions (stochastic). The goal is to learn π^* that maximizes cumulative expected reward $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$.

Chapter 4

Generalization, Overfitting, and the Bias-Variance Tradeoff

Based on Lectures of January 24 and January 28, 2026 — Lecture Slides: MLBackGround.pdf, MLBasics.pdf

4.1 The Importance of Data in ML

- Data is the foundation of all ML systems.
- **Data quality** directly determines model quality ("garbage in, garbage out").
- AI advancement is driven by three factors: **algorithms**, **data**, and **compute**.

4.2 Training Data vs. Test Data

Definition

Training data is the data used to fit (train) the model.
Test data is held-out data used to evaluate the model's performance on unseen examples.

Key Concept

A model must **never** see the test data during training. Test data serves as a proxy for real-world performance on new, unseen data.

4.3 Generalization

Definition

Generalization is the ability of a machine learning model to perform well on new, unseen data — not just the data it was trained on.

Analogy (from lecture): Consider a photo of a cat. Even if the photo is taken from a different angle or with different lighting, a human can still recognize it as a cat. A

good ML model should be able to do the same — *generalize* beyond the specific training examples.

Analogy 2 (from lecture): A student who only memorizes specific questions and answers for an exam may perform well on practice tests but poorly on the actual exam if the questions differ. This is analogous to *overfitting*.

4.4 Overfitting and Underfitting

Definition

Overfitting: The model is too complex and fits the training data too closely, including its noise. It performs well on training data but poorly on new data.

Definition

Underfitting: The model is too simple and fails to capture the underlying patterns in the data. It performs poorly on both training and test data.

	Training Error	**Test Error**
Underfitting	High	High
Good Fit	Low	Low
Overfitting	Very Low	High

Exam Tip

The hallmark of overfitting is a **large gap** between training error (low) and test error (high). A perfectly memorized training set gives zero training error but poor generalization.

4.5 The Bias-Variance Tradeoff

Definition

Bias: The error introduced by the model's simplifying assumptions. High bias $\Rightarrow$ the model misses relevant patterns $\Rightarrow$ underfitting.

Definition

Variance: The error introduced by the model's sensitivity to fluctuations in the training data. High variance $\Rightarrow$ the model captures noise $\Rightarrow$ overfitting.

Decomposition of Expected Prediction Error:

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error} \tag{4.1}$$

where:

$$\text{Bias} = \mathbb{E}[\hat{f}(x)] - f(x) \tag{4.2}$$

$$\text{Variance} = \mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right] \tag{4.3}$$

$\hat{f}(x)$ is the model's prediction, $f(x)$ is the true function, and the irreducible error is the inherent noise in the data.

Key Concept

The **goal** of ML is to find a model that minimizes the total error by **balancing** bias and variance. As model complexity increases: bias decreases but variance increases.

Model Complexity	Bias	Variance
Too simple	High	Low
Optimal	Balanced	Balanced
Too complex	Low	High

4.6 Mean Squared Error (MSE)

Mean Squared Error:

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 \tag{4.4}$$

where m is the number of samples, y_i is the true value, and $\hat{y}_i$ is the predicted value.

- MSE is the most common loss function for regression tasks.
- The main goal of an ML algorithm is to **minimize the MSE** on unseen data, not just on training data.
- Low training MSE + high test MSE $\Rightarrow$ overfitting.

4.7 Goodness of Fit

- A model's **goodness of fit** describes how well it captures the true underlying relationship.
- Neither overfitting (too much fit) nor underfitting (too little fit) is desirable.
- The optimal model complexity can be found by monitoring performance on a **validation set**.

4.8 Responsible AI and Explainability

Key Concept

Explainability is the ability to understand *why* a model makes a particular prediction. Many powerful models (deep neural networks) are "black boxes" — they achieve high accuracy but their decision-making process is opaque.

- Responsible AI requires models that are **fair**, **explainable**, and **accountable**.
- There is often a tension between model accuracy and explainability.
- Global AI governance and regulation are emerging concerns.

4.9 Practice Questions with Solutions

1. **Define generalization. Why is it the central goal of machine learning?**

   Solution: **Generalization** is a model's ability to perform well on new, unseen data. It is the central goal because a model that only works on training data is useless in practice — we deploy models to make predictions on future data they have never seen. A model that memorizes training data (overfits) fails to generalize. The entire ML pipeline (cross-validation, regularization, etc.) is designed to maximize generalization.

2. **Explain the bias-variance tradeoff with a diagram showing training/test error vs. model complexity.**

   Solution: As model complexity increases: **training error** monotonically decreases (the model fits training data better). **Test error** initially decreases (better fit) then *increases* (overfitting). The U-shaped test error curve has its minimum at the optimal complexity. At this point: **Bias** (error from simplifying assumptions) is balanced with **Variance** (error from sensitivity to training data fluctuations). Mathematically: $\text{MSE} = \text{Bias}^2 + \text{Variance} + \sigma^2_{\text{noise}}$. Simple models $\Rightarrow$ high bias, low variance. Complex models $\Rightarrow$ low bias, high variance.

3. **A model achieves 99% accuracy on training data but 60% on test data. What is the likely problem? How would you address it?**

   Solution: This is classic **overfitting** — the large gap (39%) between training and test accuracy indicates the model memorized training data including noise. Remedies:

   - **Regularization:** Add L1/L2 penalty to constrain model complexity.
   - **Reduce model complexity:** Use fewer features, shallower trees, fewer parameters.
   - **More training data:** Larger datasets reduce overfitting.
   - **Cross-validation:** Use k-fold CV to detect overfitting early.
   - **Early stopping:** Halt training when validation error begins increasing.

- **Dropout / ensemble methods:** For neural networks or tree ensembles.

4. **Derive the MSE decomposition into bias^2 + variance + irreducible error.**

   Solution: Let $y = f(x) + \epsilon$ where $\mathbb{E}[\epsilon] = 0$, $\text{Var}(\epsilon) = \sigma^2$. Let $\hat{f}(x)$ be the model prediction.

$$\begin{aligned}\text{MSE} &= \mathbb{E}[(y - \hat{f}(x))^2] = \mathbb{E}[(f(x) + \epsilon - \hat{f}(x))^2] \\ &= \mathbb{E}[(f(x) - \hat{f}(x))^2] + 2\mathbb{E}[(f(x) - \hat{f}(x))\epsilon] + \mathbb{E}[\epsilon^2]\end{aligned}$$

   Since ϵ is independent of $\hat{f}$, the cross-term vanishes. For the first term, add and subtract $\mathbb{E}[\hat{f}(x)]$:

$$\mathbb{E}[(f(x) - \hat{f}(x))^2] = (f(x) - \mathbb{E}[\hat{f}(x)])^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$

   Therefore: $\boxed{\text{MSE} = \underbrace{(f(x) - \mathbb{E}[\hat{f}(x)])^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible}}}$

5. **What is the difference between training data and test data? Why must they be kept separate?**

   Solution: **Training data** is used to fit model parameters. **Test data** is held out to evaluate generalization on unseen examples. They must be separate because evaluating on training data gives an *overly optimistic* estimate — a model can achieve zero training error by memorization. The test set acts as a proxy for future real-world data. If the model "sees" test data during training (data leakage), the evaluation is invalid.

6. **Why is explainability important in AI? Give an example where a "black box" model could cause harm.**

   Solution: Explainability allows stakeholders to understand *why* a model makes a decision, enabling trust, debugging, and accountability. *Example:* A deep neural network denies a loan application. Without explainability, the applicant cannot know why, the bank cannot verify fairness, and regulators cannot audit for discrimination. The model might be using zip code as a proxy for race (illegal redlining). Explainable models (like decision trees or LIME explanations) reveal such issues.

Chapter 5

Cross-Validation and Regularization

Based on Lectures of February 4 and February 7, 2026

5.1 Cross-Validation

Definition

Cross-validation is a technique for evaluating the performance of a machine learning model by dividing the available data into training and testing subsets, then rotating these subsets to ensure every data point is used for both training and testing.

5.1.1 Holdout Method

- Split data into a single training set and a single test set (e.g., 70/30 or 80/20).
- Simple but can have high variance — results depend on the specific split.
- This is a **non-exhaustive** method.

5.1.2 K-Fold Cross-Validation

Key Concept

In k-**fold cross-validation**, the data is partitioned into k equal-sized subsets (folds). The model is trained k times, each time using $k-1$ folds for training and the remaining fold for testing. The final performance is the average across all k iterations.

- Common choices: $k = 5$ or $k = 10$.
- Every data point appears in exactly one test fold.
- Provides a more reliable estimate than a single holdout split.

Algorithm 1 k-Fold Cross-Validation

Partition data D into k equal-sized folds $D_1, D_2, \ldots, D_k$
for $i = 1$ to k **do**
    Set test set $= D_i$
    Set training set $= D \setminus D_i$
    Train model on training set
    Evaluate model on test set $\rightarrow$ score$_i$
end for
return Average score $= \frac{1}{k} \sum_{i=1}^{k}$ score$_i$

5.1.3 Leave-One-Out Cross-Validation (LOOCV)

Definition

LOOCV is a special case of k-fold cross-validation where $k = n$ (the number of data points). Each iteration uses one data point as the test set and the remaining $n - 1$ points for training.

- Most thorough form of cross-validation.
- Particularly useful for **small (sparse) datasets** where data is precious.
- Computationally expensive for large datasets.
- Low bias but can have high variance.

5.1.4 Nested Cross-Validation

Key Concept

Nested cross-validation uses **two loops**: an **outer loop** for evaluating model performance and an **inner loop** for hyperparameter tuning. This prevents information leakage from the test set into the model selection process.

- **Outer loop:** Splits data into training + validation vs. test sets (for final evaluation).
- **Inner loop:** Within each outer fold's training set, performs k-fold CV to select optimal hyperparameters.
- This provides an **unbiased** estimate of the model's generalization performance.

Exam Tip

Nested cross-validation is essential when you are both **selecting a model** (or tuning hyperparameters) **and** evaluating its performance. Using the same data for both leads to overly optimistic estimates.

5.2 Regularization

Definition

Regularization is a technique used to prevent overfitting by adding a penalty term to the loss function. This penalty discourages the model from becoming too complex (having large weights).

Regularized Loss Function:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \cdot R(\mathbf{w}) \tag{5.1}$$

where:

- $\mathcal{L}_{\text{data}}$ is the data-fitting loss (e.g., MSE)
- $\lambda > 0$ is the regularization strength (hyperparameter)
- $R(\mathbf{w})$ is the regularization penalty

5.2.1 Types of Regularization

- **L1 Regularization (Lasso):** $R(\mathbf{w}) = \sum_j |w_j|$
  - Encourages sparsity — drives some weights to exactly zero.
  - Performs implicit feature selection.
- **L2 Regularization (Ridge):** $R(\mathbf{w}) = \sum_j w_j^2$
  - Shrinks all weights toward zero but rarely sets them to exactly zero.
  - Prevents any single feature from dominating.

5.2.2 Occam's Razor Principle

Key Concept

Occam's Razor: Among competing models that explain the data equally well, prefer the **simplest** one. In ML, this means favoring models with fewer parameters or smaller weight magnitudes.

5.2.3 Early Stopping

- Monitor the model's performance on a validation set during training.
- Stop training when validation error begins to **increase** (even if training error continues to decrease).
- Prevents the model from overfitting by limiting training iterations.

5.3 Feature Selection

Definition

Feature selection is the process of selecting a subset of the most relevant features for model construction. Not all features are useful; irrelevant features can add noise and reduce performance.

5.3.1 Methods of Feature Selection

1. **Filter Methods:** Select features based on statistical properties (e.g., correlation, mutual information) **independently** of the learning algorithm.
   - Fast and computationally cheap.
   - Example: Select features with highest correlation to the target variable.
2. **Wrapper Methods:** Evaluate subsets of features by **training a model** and assessing its performance.
   - More accurate but computationally expensive.
   - Example: Forward selection, backward elimination.
3. **Embedded Methods:** Feature selection is performed **during** model training.
   - Example: L1 regularization (Lasso) inherently selects features.
   - Example: Decision tree feature importance.

Harith's Insight / Class Context

From handwritten notes: Filter methods are fast but may miss feature interactions. Wrapper methods capture interactions but are slow. Embedded methods strike a balance by integrating selection into training.

5.4 Practice Questions with Solutions

1. **Explain k-fold cross-validation. Why is it preferred over a simple holdout split?**

   Solution: Data is partitioned into k equal folds. The model trains on $k-1$ folds, tests on the remaining fold, and this rotates k times. Final performance = average across folds. It is preferred because: (1) every data point is used for both training and testing, reducing variance; (2) a single holdout split may be unrepresentative (lucky or unlucky split); (3) k-fold gives a more reliable and stable performance estimate.

2. **What is LOOCV? When is it most appropriate to use it?**

   Solution: **Leave-One-Out Cross-Validation** is k-fold CV with $k = n$ (number of samples). Each iteration trains on $n-1$ points and tests on 1 point. Most appropriate for **small/sparse datasets** where every data point is valuable. Advantages:

nearly unbiased estimate, maximum use of data. Disadvantage: computationally expensive (n model fits) and can have high variance since test sets differ by only one point.

3. **Why is nested cross-validation necessary for hyperparameter tuning? What problem does it solve?**

   Solution: If we use the same CV loop to both select hyperparameters and estimate performance, the performance estimate is **optimistically biased** — we've effectively "tuned" to the test folds. Nested CV uses an **outer loop** for unbiased performance estimation and an **inner loop** for hyperparameter selection. This prevents information leakage from test data into the model selection process, giving a truly unbiased generalization estimate.

4. **Write the regularized loss function for L2 regularization. What does increasing λ do?**

   Solution:

$$\mathcal{L}_{\text{reg}} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 + \lambda\sum_{j=1}^{d} w_j^2$$

   Increasing λ **decreases model complexity**: the penalty on large weights grows, forcing the model toward smaller (more conservative) weights. Very large $\lambda \Rightarrow$ all weights shrink toward zero $\Rightarrow$ underfitting. $\lambda = 0 \Rightarrow$ no regularization $\Rightarrow$ potential overfitting.

5. **Compare L1 and L2 regularization. Which one performs feature selection, and why?**

   Solution:

	L1 (Lasso)	**L2 (Ridge)**
Penalty	$\sum \lvert w_j \rvert$	$\sum w_j^2$
Effect	Drives some weights to **exactly zero**	Shrinks all weights, none exactly ze
Feature selection?	**Yes**	No
Geometry	Diamond constraint region	Circular constraint region

   L1 performs feature selection because the diamond-shaped constraint region has corners on the axes. The optimal solution often lies at a corner where one or more weights are exactly zero, effectively removing those features.

6. **Explain the Occam's Razor principle in the context of machine learning.**

   Solution: Occam's Razor states: among models that explain the data equally well, prefer the **simplest** one. In ML, this means: (1) prefer fewer parameters; (2) prefer smaller weight magnitudes; (3) prefer lower-degree polynomials. Simpler models are less likely to overfit and more likely to generalize. Regularization is a direct implementation of Occam's Razor — it penalizes complexity.

7. **Compare filter, wrapper, and embedded methods of feature selection.**

   Solution:

- **Filter:** Rank features by statistical criteria (correlation, mutual information) *independently* of the model. Fast but ignores feature interactions. Example: select top-k features by correlation with target.
- **Wrapper:** Evaluate feature subsets by training a model and measuring performance. Captures interactions but computationally expensive. Example: forward selection, backward elimination.
- **Embedded:** Feature selection occurs *during* model training. Balances speed and interaction-awareness. Example: L1 regularization (Lasso), decision tree feature importance.

8. **A model has high training accuracy but low test accuracy. Suggest three techniques.**

   Solution: This is overfitting. Three techniques: (1) **Regularization** (L1/L2) to penalize complexity. (2) **Cross-validation** (k-fold) to detect and avoid overfitting during model selection. (3) **Feature selection/reduction** to remove irrelevant or noisy features (e.g., PCA, Lasso). Additional options include early stopping, collecting more training data, or using simpler models.

Chapter 6

Data Preprocessing and Feature Engineering

Based on Lectures of February 4 and February 11, 2026

6.1 The CRISP-DM Framework

Definition

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a six-phase framework for data mining and ML projects:

1. **Business Understanding:** Define objectives and requirements.
2. **Data Understanding:** Collect and explore data.
3. **Data Preparation:** Clean, transform, and engineer features.
4. **Modeling:** Train and tune models.
5. **Evaluation:** Assess model performance against objectives.
6. **Deployment:** Deploy model into production.

6.2 Data Preprocessing Pipeline

Data preprocessing is a **crucial** step in the ML lifecycle. The quality of the model depends directly on the quality of the preprocessed data.

6.2.1 Step 1: Data Profiling

- Analyze data to understand its features, trends, distributions, and ranges.
- Identify missing values, duplicate values, and outliers.
- Examine feature types: **quantitative** (numerical) vs. **categorical** (discrete classes).

6.2.2 Step 2: Data Cleansing

- Handle **missing values**:
  - **Ignore/Delete:** Remove rows with missing values (risky if many rows are affected).
  - **Impute:** Replace with mean, median, or mode of the feature.
  - **Model-based imputation:** Use a predictive model to estimate missing values.
- Remove **duplicate** entries.
- Handle **outliers:** Remove, cap, or transform extreme values.

6.2.3 Step 3: Data Reduction (Dimensionality Reduction)

Definition

Dimensionality reduction is the process of transforming data from a high-dimensional space to a lower-dimensional space while preserving the most important information. Not every dimension is equally important for explaining the domain.

- **Principal Component Analysis (PCA):** Projects data onto directions of maximum variance.
  - Compute eigenvectors and eigenvalues of the covariance matrix.
  - Select the top k eigenvectors (principal components) that explain the most variance.
  - $X = U\Sigma V^T$ (Singular Value Decomposition).
- Other techniques: t-SNE, autoencoders.

6.2.4 Step 4: Data Transformation

Feature Scaling

Features with different ranges can bias the model. Scaling brings all features to a comparable range.

Standardization (Z-score normalization):

$$x' = \frac{x - \mu}{\sigma} \tag{6.1}$$

where μ is the mean and σ is the standard deviation.

Min-Max Normalization:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{6.2}$$

Scales features to the range $[0, 1]$.

Feature Encoding

- **Label Encoding:** Assigns each category an integer (e.g., Red → 0, Blue → 1, Green → 2).
  - **Caution:** Introduces an artificial ordinal relationship between categories.
- **One-Hot Encoding:** Creates a binary column for each category.
  - No artificial ordering is introduced.
  - Can increase dimensionality significantly for features with many categories.

6.2.5 Step 5: Data Enrichment

- Combine existing features to create new, more informative ones.
- Example: Convert latitude and longitude to distance from a central reference point.
- **Polynomial features:** Create interaction terms (e.g., $x_1 \cdot x_2$, x_1^2).
- **Power transformations:** Box-Cox transformation, log transformation, binning.

Key Concept

Apply Occam's Razor to feature engineering: do not create overly complex features that lead to overfitting. Simpler features are preferred if they explain the data adequately.

6.2.6 Step 6: Data Validation

- Verify that preprocessing steps have been correctly applied.
- Ensure data is in the correct format for the chosen model.
- Check for **data leakage**.

6.3 Data Leakage

Definition

Data leakage occurs when information from the test set is inadvertently used during training. This leads to overly optimistic performance estimates that do not reflect real-world performance.

Exam Tip

Common sources of data leakage:

- Fitting the scaler (computing mean/std) on the entire dataset *before* splitting into train/test.
- Using future information (time series) in training data.
- Including the target variable (or a proxy) as a feature.

Solution: Always fit preprocessing on **training data only**, then transform both training and test data.

6.4 Pipelines

Key Concept

Pipelines automate the sequence of data preprocessing and modeling steps. They ensure that transformations are consistently applied and that **data leakage is prevented** by fitting transformations only on the training data.

6.5 Practice Questions with Solutions

1. **Describe the six phases of the CRISP-DM framework.**

   Solution: (1) **Business Understanding:** Define the problem, objectives, and requirements. (2) **Data Understanding:** Collect, explore, and assess data quality. (3) **Data Preparation:** Clean, transform, engineer features, handle missing values. (4) **Modeling:** Select algorithms, train models, tune hyperparameters. (5) **Evaluation:** Assess model against business objectives, not just metrics. (6) **Deployment:** Integrate model into production, monitor performance.

2. **What is the difference between standardization and min-max normalization? When would you prefer each?**

   Solution: **Standardization** ($x' = (x - \mu)/\sigma$) centers data at mean 0 with unit variance. **Min-max** ($x' = (x - x_{\min})/(x_{\max} - x_{\min})$) scales to $[0, 1]$. Prefer **standardization** when: data has outliers (min-max is sensitive to them), algorithm assumes normally distributed features (e.g., SVM, logistic regression). Prefer **min-max** when: you need bounded values (e.g., image pixels to $[0, 1]$), no significant outliers, algorithm requires specific range (e.g., neural networks with sigmoid).

3. **Explain one-hot encoding vs. label encoding. When would label encoding be inappropriate?**

   Solution: **Label encoding:** assigns integers (Red= 0, Blue= 1, Green= 2). **One-hot encoding:** creates binary columns (Red= $[1, 0, 0]$, Blue= $[0, 1, 0]$, Green= $[0, 0, 1]$). Label encoding is **inappropriate for nominal (unordered) categories** because it introduces a false ordinal relationship ($0 < 1 < 2$ implies Red $<$ Blue $<$

Green, which is meaningless). Distance-based algorithms (KNN, SVM) will misinterpret these as meaningful orderings. Label encoding is acceptable for truly ordinal features (e.g., Low= 0, Medium= 1, High= 2).

4. **What is data leakage? Give an example and explain how to prevent it.**

   Solution: **Data leakage** occurs when information from the test set contaminates the training process, producing overly optimistic performance estimates. *Example:* Computing the mean and standard deviation for standardization on the *entire* dataset before splitting into train/test. The training set then contains statistical information about the test set. *Prevention:* Always fit preprocessing (scaler, imputer, encoder) on **training data only**, then apply the fitted transform to both train and test. Use **pipelines** to automate this.

5. **A dataset has 100 features. Describe how you would use PCA to reduce dimensionality.**

   Solution: (1) Standardize all features (mean 0, variance 1). (2) Compute the 100×100 covariance matrix. (3) Compute eigenvalues and eigenvectors. (4) Sort eigenvectors by decreasing eigenvalue. (5) Select top k eigenvectors (principal components) that capture a desired fraction of total variance (e.g., 95%). (6) Project data onto these k components: $X_{\text{reduced}} = X \cdot W_k$ where W_k is the matrix of top k eigenvectors. Result: dimensionality reduced from 100 to k.

6. **Why is it important to fit the scaler on training data only?**

   Solution: If the scaler is fit on the entire dataset (including test data), the training set incorporates statistics (mean, std, min, max) from the test set. This is **data leakage** — the model indirectly "sees" test data during training. Performance estimates become unrealistically optimistic and do not reflect real-world deployment where future data statistics are unknown.

7. **What is a pipeline in ML? How does it help prevent data leakage?**

   Solution: A **pipeline** chains preprocessing steps and model training into a single object. When used with cross-validation, the pipeline ensures that at each fold: (1) preprocessing (scaling, encoding, imputation) is fit *only* on the training fold; (2) the fitted transform is applied to the test fold. This prevents data leakage because no test-fold statistics leak into preprocessing. Example in scikit-learn: `Pipeline([('scaler', StandardScaler()), ('model', LogisticRegression())])`.

Chapter 7

Data Mining and Association Rules

Based on Lectures of February 18 and February 21, 2026 — Lecture Slides: gujjar_ 5.pdf, gujjar_ 6.pdf

7.1 Data Science, Data Analytics, and Data Mining

Definition

Data Science is an interdisciplinary field that uses statistics, scientific computing, and scientific methods to extract knowledge from data.

Definition

Data Analytics is the process of analyzing and visualizing data to extract insights and make informed decisions.

Definition

Data Mining is the process of discovering valuable patterns, relationships, and information from large datasets using algorithms and statistical methods.

Key Concept

Data mining $\subset$ Data science. Data mining focuses on *discovering* patterns, while data analytics focuses on *using* those patterns to make decisions. Data science encompasses both, plus broader activities like data engineering and deployment.

7.2 Association Rules

Definition

An **association rule** is a pattern of the form $X \to Y$, where X and Y are sets of items (itemsets), and $X \cap Y = \emptyset$. The rule states: "If X is present, then Y is also likely to be present."

Classic example (Market Basket Analysis): "Customers who buy milk are also likely to buy bread" — represented as $\{\text{milk}\} \to \{\text{bread}\}$.

7.2.1 Key Measures for Association Rules

Support: The proportion of transactions that contain both X and Y.

$$\text{Support}(X \to Y) = \frac{|\{t \in D \mid X \cup Y \subseteq t\}|}{|D|} \tag{7.1}$$

Confidence: The proportion of transactions containing X that also contain Y.

$$\text{Confidence}(X \to Y) = \frac{|\{t \in D \mid X \cup Y \subseteq t\}|}{|\{t \in D \mid X \subseteq t\}|} = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)} \tag{7.2}$$

Lift: Measures the strength of the association, accounting for the base frequency of Y.

$$\text{Lift}(X \to Y) = \frac{\text{Confidence}(X \to Y)}{\text{Support}(Y)} = \frac{P(X \cap Y)}{P(X) \cdot P(Y)} \tag{7.3}$$

Key Concept

- **Lift** > 1: Positive association — X and Y appear together more often than expected.
- **Lift** $= 1$: No association — X and Y are independent.
- **Lift** < 1: Negative association — X and Y appear together less often than expected.

Exam Tip

For a rule to be considered "interesting," it must satisfy both **minimum support** and **minimum confidence** thresholds. Additionally, check the lift to ensure the association is meaningful (lift > 1).

7.2.2 The Apriori Algorithm

Definition

The **Apriori Algorithm** is an iterative algorithm for mining frequent itemsets and generating association rules. It uses the **Apriori property**: any subset of a frequent itemset must also be frequent.

Harith's Insight / Class Context

From handwritten notes: The Apriori algorithm works well with **sparse data**. In dense datasets (where many items co-occur frequently), the number of candidates can explode. The key pruning principle is: if an itemset is infrequent, all its supersets are also infrequent.

Algorithm 2 Apriori Algorithm (Simplified)

```
Generate all singleton itemsets with support ≥ min_support
k ← 1
repeat
    Generate candidate (k + 1)-itemsets from frequent k-itemsets
    Scan database to count support of each candidate
    Prune candidates below min_support
    k ← k + 1
until no new frequent itemsets are found
Generate rules from frequent itemsets satisfying min_confidence
```

7.3 Practice Questions with Solutions

1. **What is the difference between data mining and data analytics?**

   Solution: **Data mining** is the process of *discovering* patterns, relationships, and anomalies in large datasets using algorithms (e.g., association rules, clustering). **Data analytics** is the process of *using* those discovered patterns to make informed decisions, often through visualization and statistical analysis. Data mining asks "What patterns exist?" while analytics asks "What should we do about them?"

2. **Compute support, confidence, and lift for {bread} → {milk}.**

   Solution: Given: $|D| = 100$, bread in 40, milk in 30, both in 20.

$$\text{Support} = \frac{|\text{bread} \cap \text{milk}|}{|D|} = \frac{20}{100} = 0.20 \quad (20\%)$$

$$\text{Confidence} = \frac{|\text{bread} \cap \text{milk}|}{|\text{bread}|} = \frac{20}{40} = 0.50 \quad (50\%)$$

$$\text{Lift} = \frac{\text{Confidence}}{\text{Support(milk)}} = \frac{0.50}{30/100} = \frac{0.50}{0.30} = 1.67$$

   Lift > 1 indicates a **positive association**: buying bread increases the likelihood of buying milk beyond what random chance would predict.

3. **Explain the Apriori property and how it is used for pruning.**

   Solution: The **Apriori property** (downward closure): *any subset of a frequent itemset must also be frequent.* Equivalently (contrapositive): if an itemset is infrequent, all its supersets are also infrequent. **Pruning:** When generating candidate $(k+1)$-itemsets, if *any* k-subset of a candidate is not in the frequent k-itemsets, the candidate is pruned without scanning the database. This dramatically reduces the search space.

4. **When is a lift value of exactly 1 significant? What does it tell you?**

   Solution: Lift $= 1$ means $P(Y \mid X) = P(Y)$ — knowing X gives **no information** about Y. The items are **statistically independent**. This is significant because even if support and confidence appear high, the association is *spurious* — Y occurs just as often without X. A lift of 1 means the rule has no predictive value.

5. **{laptop} → {printer} has high support and confidence but lift < 1. What does this mean?**

   Solution: Lift < 1 indicates a **negative association**: buying a laptop actually *decreases* the probability of buying a printer compared to the baseline. The high confidence is misleading because printers are already very commonly purchased (high $P(\text{printer})$). The rule is **not useful** — the apparent association is weaker than random chance. This demonstrates why lift is essential: support and confidence alone can be misleading.

Chapter 8

Classification

Based on Lectures of February 21 and March 7, 2026

8.1 What is Classification?

Definition

Classification is a supervised learning task where the goal is to learn a function $f : X \to Y$ that maps input features $X = \mathbb{R}^d$ to a discrete set of class labels $Y = \{1, 2, \ldots, k\}$.

- The function f is called a **classifier**.
- Given a **training set** of labeled examples $\{(x_i, y_i)\}_{i=1}^n$, the classifier learns to predict y for new inputs x.
- The primary goal is to **minimize the error rate** (number of misclassified instances).

8.2 Evaluating a Classifier

8.2.1 Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

8.2.2 Performance Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{8.1}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{(Of all predicted positives, how many are correct?)} \tag{8.2}$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad \text{(Of all actual positives, how many were found?)} \tag{8.3}$$

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \tag{8.4}$$

Additional Rates:

$$\text{False Positive Rate (FPR)} = \frac{FP}{FP + TN} \tag{8.5}$$

$$\text{False Negative Rate (FNR)} = \frac{FN}{FN + TP} \tag{8.6}$$

$$\text{True Negative Rate (Specificity)} = \frac{TN}{TN + FP} \tag{8.7}$$

Exam Tip

Accuracy is not always the best metric. In imbalanced datasets (e.g., 99% negative, 1% positive), a classifier that always predicts "negative" achieves 99% accuracy but is useless. Use **precision, recall, and F1 score** for imbalanced problems.

Key Concept

The **F1 Score** is the **harmonic mean** of precision and recall. It balances both metrics and is particularly useful when classes are imbalanced. A high F1 score requires both high precision and high recall.

8.3 Bayes Classifier

Definition

The **Bayes Classifier** uses Bayes' Theorem to classify inputs by computing the **posterior probability** of each class given the input features, and assigning the class with the highest posterior.

8.3.1 Bayes' Theorem

Bayes' Theorem:

$$P(Y = k \mid X = x) = \frac{P(X = x \mid Y = k) \cdot P(Y = k)}{P(X = x)} \tag{8.8}$$

where:

- $P(Y = k)$ is the **prior** probability of class k
- $P(X = x \mid Y = k)$ is the **likelihood** of observing x given class k
- $P(Y = k \mid X = x)$ is the **posterior** probability of class k given x
- $P(X = x)$ is the **evidence** (normalizing constant)

Optimal Bayes Classification:

$$\hat{y} = \arg\max_k P(Y = k \mid X = x) = \arg\max_k P(X = x \mid Y = k) \cdot P(Y = k) \tag{8.9}$$

Note: $P(X = x)$ does not depend on k, so it plays no role in the decision.

8.3.2 Challenge: The Curse of Dimensionality

Key Concept

With d binary features, estimating the full likelihood $P(X \mid Y)$ requires specifying 2^d probabilities for each class. This grows **exponentially** with the number of features — the **curse of dimensionality**.

8.3.3 Naïve Bayes Classifier

Definition

The **Naïve Bayes Classifier** assumes that all features are **conditionally independent** given the class label. This dramatically simplifies the likelihood computation.

Naïve Bayes assumption:

$$P(X = x \mid Y = k) = \prod_{j=1}^{d} P(X_j = x_j \mid Y = k) \tag{8.10}$$

Naïve Bayes classification:

$$\hat{y} = \arg\max_k P(Y = k) \prod_{j=1}^{d} P(X_j = x_j \mid Y = k) \tag{8.11}$$

- Reduces parameter estimation from 2^d to $2d$ per class (for binary features).

- The independence assumption is often "naïve" (violated in practice), but Naïve Bayes works surprisingly well in many real-world applications.
- Particularly effective for text classification (spam filtering, sentiment analysis).

Harith's Insight / Class Context

From handwritten notes: The key insight is that the independence assumption *trades accuracy in probability estimation for tractability.* Even when the assumption is wrong, the *ranking* of posterior probabilities (which is what matters for classification) can still be correct.

8.4 Generative vs. Discriminative Models

Key Concept

- **Generative models** learn the **joint distribution** $P(X, Y)$ and use Bayes' rule to compute $P(Y \mid X)$. Examples: Naïve Bayes, Hidden Markov Models.
- **Discriminative models** learn the **conditional distribution** $P(Y \mid X)$ directly. Examples: Logistic Regression, SVMs, Decision Trees.

8.5 Practice Questions with Solutions

1. **Define accuracy, precision, recall, and F1 score. Why is accuracy insufficient for imbalanced datasets?**

   Solution: **Accuracy** $= (TP + TN)/(TP + TN + FP + FN)$: fraction of correct predictions. **Precision** $= TP/(TP + FP)$: of all predicted positives, how many are truly positive. **Recall** $= TP/(TP + FN)$: of all actual positives, how many were detected. **F1** $= 2 \cdot \text{Prec} \cdot \text{Rec}/(\text{Prec} + \text{Rec})$: harmonic mean of precision and recall. In imbalanced datasets (e.g., 99% negative), a classifier predicting "always negative" achieves 99% accuracy but 0% recall — completely useless for detecting the minority class.

2. **Derive the Naïve Bayes classification rule.**

   Solution: Start with Bayes' Theorem:

   $$P(Y = k \mid X = x) = \frac{P(X = x \mid Y = k) \cdot P(Y = k)}{P(X = x)}$$

   The denominator $P(X = x)$ is constant across classes, so:

   $$\hat{y} = \arg\max_k P(X = x \mid Y = k) \cdot P(Y = k)$$

   Apply the **conditional independence assumption** — features are independent given the class:

   $$P(X = x \mid Y = k) = \prod_{j=1}^{d} P(X_j = x_j \mid Y = k)$$

Therefore:

$$\boxed{\hat{y} = \arg\max_k P(Y = k) \prod_{j=1}^{d} P(X_j = x_j \mid Y = k)}$$

3. **A medical test has 95% sensitivity and 90% specificity. Disease prevalence is 1%. Find $P(\text{disease} \mid \text{positive test})$.**

   Solution: Let D = disease, $+$ = positive test. Given: $P(+ \mid D) = 0.95$ (sensitivity), $P(- \mid \neg D) = 0.90$ (specificity, so $P(+ \mid \neg D) = 0.10$), $P(D) = 0.01$.

$$\begin{aligned} P(D \mid +) &= \frac{P(+ \mid D) \cdot P(D)}{P(+)} \\ P(+) &= P(+ \mid D)P(D) + P(+ \mid \neg D)P(\neg D) \\ &= 0.95 \times 0.01 + 0.10 \times 0.99 = 0.0095 + 0.099 = 0.1085 \\ P(D \mid +) &= \frac{0.0095}{0.1085} \approx \boxed{0.0876 \approx 8.8\%} \end{aligned}$$

   Despite a "95% accurate" test, the probability of disease given a positive result is only $\sim$8.8%, because the disease is rare ($P(D) = 1\%$).

4. **What is the curse of dimensionality? How does Naïve Bayes address it?**

   Solution: The **curse of dimensionality**: as the number of features d increases, the number of parameters to estimate the full joint distribution $P(X \mid Y)$ grows **exponentially** (2^d for binary features per class). With limited data, reliable estimation becomes impossible. **Naïve Bayes** addresses this by assuming conditional independence, reducing the parameter count from 2^d to $2d$ per class (for binary features). Each feature's distribution is estimated independently, making estimation tractable even for high-dimensional data.

5. **Compare generative and discriminative models. Give an example of each.**

   Solution:

   - **Generative:** Models the *joint distribution* $P(X, Y)$, then uses Bayes' rule to get $P(Y \mid X)$. Can generate synthetic data. Example: **Naïve Bayes**, Gaussian Mixture Models, Hidden Markov Models.
   - **Discriminative:** Models $P(Y \mid X)$ directly (or learns a decision boundary). Cannot generate data. Example: **Logistic Regression**, SVMs, Decision Trees, Neural Networks.

   Discriminative models often achieve higher classification accuracy; generative models are useful when you need to model the data distribution or handle missing features.

6. **Given $TP = 80, FP = 10, FN = 20, TN = 90$, compute all metrics.**

Solution:

$$\text{Accuracy} = \frac{80 + 90}{80 + 90 + 10 + 20} = \frac{170}{200} = \boxed{0.85 = 85\%}$$

$$\text{Precision} = \frac{80}{80 + 10} = \frac{80}{90} = \boxed{0.889 \approx 88.9\%}$$

$$\text{Recall} = \frac{80}{80 + 20} = \frac{80}{100} = \boxed{0.80 = 80\%}$$

$$\text{F1} = 2 \cdot \frac{0.889 \times 0.80}{0.889 + 0.80} = 2 \cdot \frac{0.711}{1.689} = \boxed{0.842 \approx 84.2\%}$$

Chapter 9

Decision Trees

Based on Lectures of March 7 and March 11, 2026

9.1 What is a Decision Tree?

Definition

A **decision tree** is a tree-structured model for classification (or regression) where:

- Each **internal node** tests a feature (attribute).
- Each **branch** corresponds to a feature value or condition.
- Each **leaf node** assigns a class label (or continuous value).

Key Concept

Decision trees are **interpretable**: you can trace the path from root to leaf to understand *why* a particular prediction was made. This makes them valuable in applications requiring explainability.

9.2 Building a Decision Tree: Choosing the Best Feature

The central question in building a decision tree is: **which feature should be used at each node?** The answer involves selecting the feature that provides the most **information gain**.

9.2.1 Entropy

Definition

Entropy measures the **uncertainty** or **impurity** in a set of data. For a set S with k classes:

$$H(S) = -\sum_{i=1}^{k} p_i \log_2 p_i \tag{9.1}$$

where p_i is the proportion of samples belonging to class i.

Properties of Entropy:

- $H(S) = 0$ when all samples belong to one class (pure node — no uncertainty).
- $H(S) = 1$ for a binary split with equal proportions (maximum uncertainty).
- $0 \leq H(S) \leq \log_2 k$.
- Convention: $0 \cdot \log_2 0 = 0$.

Example 9.1. Consider a dataset with 14 samples: 9 positive and 5 negative.

$$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940 \text{ bits}$$

9.2.2 Information Gain

Definition

Information Gain (IG) measures the reduction in entropy achieved by splitting the data on a particular feature A:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot H(S_v) \tag{9.2}$$

where S_v is the subset of S for which feature A has value v.

Key Concept

At each node, select the feature with the **highest information gain**. This greedily minimizes the expected entropy of the child nodes, leading to the most informative split.

Example 9.2. Weather example for playing a sport: Given features {Outlook, Temperature, Humidity, Wind}, compute information gain for each feature. Suppose splitting on "Outlook" reduces entropy the most — then "Outlook" becomes the root node.

9.2.3 Gini Index (Alternative Splitting Criterion)

Gini Index:

$$\text{Gini}(S) = 1 - \sum_{i=1}^{k} p_i^2 \tag{9.3}$$

- Used in the **CART (Classification and Regression Trees)** algorithm.

- Gini $= 0$ for a pure node; maximum value approaches $1 - 1/k$ for k classes.
- Often produces similar results to information gain but is computationally simpler (no logarithm).

9.3 Decision Tree Construction Algorithm

Algorithm 3 ID3 Decision Tree Algorithm

function BUILDTREE(S, Features)
  if all samples in S have the same class c **then**
    return Leaf node with class c
  end if
  if Features is empty **then**
    return Leaf node with majority class in S
  end if
  $A^* \leftarrow \arg\max_{A \in \text{Features}} IG(S, A)$
  Create internal node for feature A^*
  for each value v of A^* **do**
    $S_v \leftarrow \{x \in S \mid x.A^* = v\}$
    if S_v is empty **then**
      Add leaf with majority class of S
    else
      Add subtree $\leftarrow$ BUILDTREE(S_v, Features $\setminus \{A^*\}$)
    end if
  end for
end function

9.3.1 Overfitting in Decision Trees

- Deep trees can perfectly memorize training data $\Rightarrow$ overfitting.
- **Pre-pruning:** Stop growing the tree early (limit depth, minimum samples per leaf).
- **Post-pruning:** Grow the full tree, then remove branches that do not improve validation performance.

9.4 Practice Questions with Solutions

1. **What is entropy? Compute the entropy of a dataset with 8 positive and 4 negative examples.**

   Solution: Entropy measures uncertainty/impurity: $H(S) = -\sum p_i \log_2 p_i$. With 8

positive ($p_+ = 8/12 = 2/3$) and 4 negative ($p_- = 4/12 = 1/3$):

$$\begin{aligned} H(S) &= -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} \\ &= -\frac{2}{3}(-0.585) - \frac{1}{3}(-1.585) \\ &= 0.390 + 0.528 = \boxed{0.918 \text{ bits}} \end{aligned}$$

2. **Define information gain. Why is it used as the splitting criterion?**

   Solution: Information gain $IG(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$ measures the **reduction in entropy** after splitting on feature A. It is used because a higher IG means the split produces purer (more homogeneous) child nodes, bringing us closer to a correct classification. By greedily choosing the feature with maximum IG at each node, we build the most informative tree.

3. **Compare entropy-based splitting (ID3) with Gini-based splitting (CART).**

   Solution:

   - **ID3 (Entropy):** $H = -\sum p_i \log_2 p_i$. Slightly more sensitive to class probabilities due to the logarithm. Can produce multi-way splits.
   - **CART (Gini):** $G = 1 - \sum p_i^2$. Computationally faster (no logarithm). Always produces binary splits.
   - In practice, both produce **similar trees**. Gini is preferred when speed matters; entropy is more information-theoretically motivated.

4. **Construct the first two levels of a decision tree using information gain.**

   Solution (template): Suppose the dataset has features {Color, Size, Shape} with binary labels. (1) Compute $H(S)$ for the entire dataset. (2) For each feature, compute $IG(S, \text{feature})$ by computing the weighted entropy of the child nodes after splitting. (3) The feature with the **highest IG** becomes the root. (4) For each branch of the root, repeat: compute IG for the remaining features on each subset, and pick the best. This gives two levels. The key is: always pick the feature that reduces entropy the most.

5. **How does overfitting manifest in decision trees? Describe pre-pruning and post-pruning.**

   Solution: Overfitting manifests as very deep trees with many leaves that perfectly classify training data (zero training error) but perform poorly on test data. Each leaf may represent very few (even single) training points, capturing noise.

   - **Pre-pruning (early stopping):** Stop tree growth before completion. Criteria: maximum depth limit, minimum samples per leaf, minimum information gain threshold.
   - **Post-pruning:** Grow the full tree first, then remove branches that don't improve performance on a validation set. More effective but computationally more expensive. Example: reduced error pruning, cost-complexity pruning.

6. **Why are decision trees considered "interpretable" models?**

   Solution: Decision trees produce explicit if-then rules that can be traced from root to leaf. A human can follow the path and understand *exactly* why a prediction was made (e.g., "If Outlook=Sunny AND Humidity=High, then Don't Play"). This is unlike neural networks or SVMs where the decision process is opaque. Interpretability is crucial in domains like medicine or law where decisions must be justified.

Chapter 10

K-Nearest Neighbors (KNN)

Based on Lecture of March 11, 2026

10.1 The KNN Algorithm

Definition

K-Nearest Neighbors (KNN) is a non-parametric, instance-based (lazy) learning algorithm. It classifies a new data point by finding the K closest training examples and assigning the most common class among them.

Algorithm 4 KNN Classification

Require: Training data $\{(x_i, y_i)\}_{i=1}^{n}$, query point x_q, integer K

1: Compute distance $d(x_q, x_i)$ for all $i = 1, \ldots, n$
2: Find the K nearest neighbors $\mathcal{N}_K(x_q)$
3: Assign class $\hat{y} =$ majority vote among $\{y_i : x_i \in \mathcal{N}_K(x_q)\}$

10.2 Distance Metrics

Euclidean Distance (for continuous features):

$$d(x, x') = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2} \tag{10.1}$$

Manhattan Distance:

$$d(x, x') = \sum_{j=1}^{d} |x_j - x'_j| \tag{10.2}$$

Minkowski Distance (generalization):

$$d(x, x') = \left(\sum_{j=1}^{d} |x_j - x'_j|^p \right)^{1/p} \tag{10.3}$$

Euclidean $= p = 2$, Manhattan $= p = 1$.
Hamming Distance (for discrete/categorical features):

$$d(x, x') = \sum_{j=1}^{d} \mathbb{1}[x_j \neq x'_j] \tag{10.4}$$

Counts the number of positions where the features differ.

Key Concept

Properties of a valid distance function:

1. **Non-negativity:** $d(x, x') \geq 0$
2. **Identity:** $d(x, x') = 0 \iff x = x'$
3. **Symmetry:** $d(x, x') = d(x', x)$
4. **Triangle inequality:** $d(x, x'') \leq d(x, x') + d(x', x'')$

10.3 Choosing K

- **Small K (e.g., $K = 1$):** Sensitive to noise, low bias, high variance $\Rightarrow$ overfitting.
- **Large K:** Smoother decision boundaries, high bias, low variance $\Rightarrow$ underfitting.
- Typically choose K using **cross-validation**.
- **Rule of thumb:** $K = \sqrt{n}$ (where n is the number of training samples).

10.4 Strengths and Weaknesses

Strengths	Weaknesses
Simple to implement and understand	Computationally expensive at prediction time (lazy learner)
No training phase needed	**Curse of dimensionality:** performance degrades in high dimensions
Can handle non-linear decision boundaries	Sensitive to irrelevant features
Works for both classification and regression	Requires feature scaling

Exam Tip

KNN is a **lazy learner**: it does no work during training and performs all computation at prediction time. This contrasts with **eager learners** (like decision trees or neural networks) that build a model during training.

10.5 Practice Questions with Solutions

1. **Describe the KNN algorithm for classification. What is the role of K?**

   Solution: Given a query point x_q: (1) Compute the distance from x_q to every training point. (2) Select the K nearest neighbors. (3) Assign x_q the **majority class** among the K neighbors. **Role of K:** Controls the bias-variance tradeoff. Small K (e.g., 1): low bias, high variance — sensitive to noise, complex decision boundary. Large K: high bias, low variance — smoother boundary, may underfit. K is typically chosen via cross-validation.

2. **Compute Euclidean and Manhattan distances between $x = (1, 2, 3)$ and $x' = (4, 0, 1)$.**

   Solution:

   $$d_{\text{Euclidean}} = \sqrt{(4-1)^2 + (0-2)^2 + (1-3)^2} = \sqrt{9+4+4} = \sqrt{17} \approx \boxed{4.123}$$

   $$d_{\text{Manhattan}} = |4-1| + |0-2| + |1-3| = 3+2+2 = \boxed{7}$$

3. **What is the curse of dimensionality, and how does it affect KNN?**

   Solution: In high dimensions, data becomes increasingly **sparse**: the volume of the feature space grows exponentially with d, so the ratio of nearest to farthest neighbor distances approaches 1. All points become roughly equidistant, making the "nearest neighbor" concept meaningless. For KNN specifically: (1) distance-based similarity breaks down; (2) exponentially more data is needed to maintain the same point density; (3) irrelevant features add noise to distance calculations.

4. **Why is feature scaling important for KNN?**

   Solution: KNN relies on distance calculations. If features have very different ranges (e.g., age $\in [0, 100]$ vs. income $\in [0, 1{,}000{,}000]$), the feature with the larger range

dominates the distance. A difference of 1 in income is negligible, but the distance calculation treats it as significant. Feature scaling (standardization or normalization) ensures all features contribute proportionally to the distance metric.

5. **Compare KNN with decision trees.**

   Solution:

	KNN	**Decision Tree**
Training time	$O(1)$ (lazy — stores data)	$O(n \cdot d \cdot \log n)$ (builds tree)
Prediction time	$O(n \cdot d)$ (slow — scans all data)	$O(\text{depth})$ (fast — traverses tree)
Interpretability	Low (no explicit model)	High (if-then rules)
Handles non-linearity	Yes (naturally)	Yes (axis-aligned splits)
Sensitive to scaling	Yes (distance-based)	No (comparisons only)

Chapter 11

Clustering and K-Means

Based on Lecture of March 11, 2026

11.1 What is Clustering?

Definition

Clustering is an **unsupervised learning** technique that groups similar data points together into clusters. The goal is to ensure that points within the same cluster are more similar to each other than to points in other clusters.

- **Hard clustering:** Each data point belongs to exactly one cluster.
- **Soft (fuzzy) clustering:** Each data point has a probability of belonging to each cluster.

11.1.1 Types of Clustering

1. **Connectivity-based (Hierarchical):**
    - **Agglomerative (bottom-up):** Start with each point as its own cluster; merge closest clusters iteratively.
    - **Divisive (top-down):** Start with all points in one cluster; split iteratively.
2. **Centroid-based:**
    - K-Means is the most popular example.

Algorithm 5 K-Means Clustering

Require: Data $\{x_1, \ldots, x_n\}$, number of clusters K

1: Randomly initialize K cluster centers (centroids) $\mu_1, \ldots, \mu_K$
2: **repeat**
3: **Assignment step:** Assign each point x_i to the nearest centroid:
4: $c_i = \arg\min_k \|x_i - \mu_k\|^2$
5: **Update step:** Recompute each centroid as the mean of its assigned points:
6: $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$
7: **until** centroids do not change (convergence)

11.2 The K-Means Algorithm

Key Concept

K-Means **minimizes within-cluster variance** (sum of squared distances from each point to its cluster center):

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \tag{11.1}$$

This is an NP-hard optimization problem; K-Means finds a **local** minimum.

Harith's Insight / Class Context

From handwritten notes: Finding the globally optimal clustering is NP-hard. K-Means uses a heuristic approach (iterative refinement) that converges but may find different solutions depending on initialization. Running K-Means multiple times with different initializations helps find a better solution.

11.3 Choosing K: Evaluation Metrics

11.3.1 Elbow Method

- Plot the within-cluster sum of squares J as a function of K.
- Look for the "elbow point" where increasing K yields diminishing returns.

11.3.2 Silhouette Score

For each data point i:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \tag{11.2}$$

where:

- $a(i)$ = average distance from i to other points in the same cluster
- $b(i)$ = average distance from i to points in the nearest other cluster

$s(i) \in [-1, 1]$. Values close to 1 indicate good clustering.

11.3.3 Davies-Bouldin Index

- Lower values indicate better clustering.
- Measures the average similarity between each cluster and its most similar cluster.

11.4 Practice Questions with Solutions

1. **Describe the K-Means algorithm step-by-step. Why is initialization important?**

   Solution: (1) Choose K and randomly initialize K centroids. (2) **Assignment:** assign each point to the nearest centroid. (3) **Update:** recompute each centroid as the mean of its assigned points. (4) Repeat steps 2–3 until convergence (centroids stop moving). **Initialization matters** because K-Means converges to a *local* minimum — different initializations can lead to very different clusterings. Bad initialization can produce poor results. Common fix: run K-Means multiple times with random initializations (or use K-Means++).

2. **What objective function does K-Means minimize? Is it guaranteed to find the global minimum?**

   Solution: K-Means minimizes the **within-cluster sum of squares (WCSS)**:

   $$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

   No, it is **not** guaranteed to find the global minimum. The optimization is NP-hard in general. K-Means uses a greedy iterative approach that is guaranteed to **converge** (objective decreases monotonically) but only to a **local** minimum.

3. **Explain the Elbow Method and Silhouette Score for choosing K.**

   Solution:

   - **Elbow Method:** Plot WCSS (J) vs. K. As K increases, J decreases. Look for the "elbow" — the point where the rate of decrease sharply slows. This K balances fit and complexity.

- **Silhouette Score:** For each point i: $s(i) = (b(i) - a(i))/\max(a(i), b(i))$ where $a(i)$ = avg distance to same-cluster points, $b(i)$ = avg distance to nearest-other-cluster points. $s \in [-1, 1]$; higher is better. Average over all points gives the overall silhouette score. Choose K that maximizes it.

4. **What is the difference between hard and soft clustering?**

   Solution: **Hard clustering:** each data point belongs to **exactly one** cluster (e.g., K-Means). **Soft (fuzzy) clustering:** each data point has a **probability** (or degree of membership) for each cluster (e.g., Gaussian Mixture Models). Soft clustering is more flexible when cluster boundaries overlap or when a point genuinely belongs partially to multiple groups.

5. **Compare agglomerative and divisive hierarchical clustering.**

   Solution:

   - **Agglomerative (bottom-up):** Start with n individual clusters (one per point). Iteratively merge the two closest clusters until only one remains. Produces a dendrogram; cut at desired level for K clusters.
   - **Divisive (top-down):** Start with all points in one cluster. Iteratively split the least cohesive cluster until n clusters remain.

   Agglomerative is more common (simpler to implement, $O(n^2 \log n)$). Divisive is $O(2^n)$ in the worst case and rarely used in practice.

6. **Run one iteration of K-Means with $K = 2$, initial centroids $(1, 1)$ and $(6, 6)$.**

   Solution: Data: $(1, 1), (1, 2), (5, 5), (6, 5), (6, 6)$.

   Assignment step (assign each point to nearest centroid):

   - $(1, 1)$: d to $(1, 1) = 0$, d to $(6, 6) = \sqrt{50} \approx 7.07 \Rightarrow$ Cluster 1
   - $(1, 2)$: d to $(1, 1) = 1$, d to $(6, 6) = \sqrt{41} \approx 6.40 \Rightarrow$ Cluster 1
   - $(5, 5)$: d to $(1, 1) = \sqrt{32} \approx 5.66$, d to $(6, 6) = \sqrt{2} \approx 1.41 \Rightarrow$ Cluster 2
   - $(6, 5)$: d to $(1, 1) = \sqrt{41} \approx 6.40$, d to $(6, 6) = 1 \Rightarrow$ Cluster 2
   - $(6, 6)$: d to $(1, 1) = \sqrt{50} \approx 7.07$, d to $(6, 6) = 0 \Rightarrow$ Cluster 2

   $C_1 = \{(1, 1), (1, 2)\}$, $C_2 = \{(5, 5), (6, 5), (6, 6)\}$.

   Update step (recompute centroids):

$$\mu_1 = \left(\frac{1+1}{2}, \frac{1+2}{2}\right) = \boxed{(1, 1.5)}$$

$$\mu_2 = \left(\frac{5+6+6}{3}, \frac{5+5+6}{3}\right) = \boxed{(5.67, 5.33)}$$

Chapter 12

Decision-Making Under Uncertainty

Based on Lectures of March 19 and March 25, 2026

12.1 Why Uncertainty Matters in AI

In real-world applications, AI agents often face situations where:

- Outcomes are not deterministic.
- Information is incomplete or noisy.
- Multiple actions are possible, each with different consequences.

Key Concept

Decision-making under uncertainty requires combining **probability theory** (to model uncertainty) with **utility theory** (to model preferences and outcomes).

12.2 Probability Theory Review

12.2.1 Basic Probability Notation

- $P(X)$: Probability of event X
- $P(X, Y) = P(X \cap Y)$: Joint probability of X and Y
- $P(X \mid Y)$: Conditional probability of X given Y
- $\sum_x P(X = x) = 1$ for any random variable X

12.2.2 Joint Probability Distribution

Definition

A **joint probability distribution** $P(X_1, X_2, \ldots, X_n)$ specifies the probability of every possible combination of values for the random variables $X_1, \ldots, X_n$.

Key Concept

The joint distribution is the "complete knowledge" about a domain. **Any** probability query can be answered from it using marginalization and conditioning.

12.2.3 Marginalization

Marginalization (Summing Out):

$$P(X) = \sum_y P(X, Y = y) \tag{12.1}$$

To compute the probability of X, sum the joint probability over all values of Y.

12.2.4 Conditional Probability and Bayes' Rule

Conditional Probability:

$$P(X \mid Y) = \frac{P(X, Y)}{P(Y)} \tag{12.2}$$

Bayes' Rule:

$$P(Y \mid X) = \frac{P(X \mid Y) \cdot P(Y)}{P(X)} \tag{12.3}$$

12.2.5 Inference Using Full Joint Distribution

- Given the full joint distribution, any conditional or marginal probability can be computed.
- Example (from lecture): In a diagnostic domain (e.g., toothache and cavity), compute $P(\text{cavity} \mid \text{toothache})$ using the joint distribution table.
- **Normalization:** When computing $P(Y \mid X)$, we can compute relative likelihoods and normalize:

  $$P(Y \mid X) = \alpha \cdot P(X \mid Y) \cdot P(Y) \tag{12.4}$$

  where $\alpha = 1/P(X)$ is the normalizing constant.

12.3 Independence and Conditional Independence

Definition

Two variables X and Y are **independent** if:

$$P(X, Y) = P(X) \cdot P(Y) \tag{12.5}$$

Equivalently, $P(X \mid Y) = P(X)$.

Definition

Two variables X and Y are **conditionally independent** given Z if:

$$P(X, Y \mid Z) = P(X \mid Z) \cdot P(Y \mid Z) \tag{12.6}$$

Equivalently, $P(X \mid Y, Z) = P(X \mid Z)$.

Key Concept

Conditional independence is one of the most important concepts in AI. It allows us to decompose large joint probability distribution tables into smaller, manageable pieces.

Example: Consider 50 variables, each with 5 possible values. The full joint distribution has 5^{50} entries — computationally infeasible. But if each variable depends on at most 5 others (given the rest), the complexity reduces from 5^{50} to something like $50 \times 5^5 = 156{,}250$ — a dramatic reduction.

Harith's Insight / Class Context

From handwritten notes: Conditional independence is **more common** than absolute independence in real-world domains. It allows probabilistic systems to "scale up." The idea of decomposing large tables into weakly connected subsets is one of the most important developments in AI.

12.4 Causal vs. Diagnostic Knowledge

Definition

Causal knowledge flows in the "natural" direction: cause → effect. It is typically documented and readily available. Example: "Meningitis causes a stiff neck."

Definition

Diagnostic knowledge flows in the "reverse" direction: effect → cause. It requires real-time inference. Example: "Given a stiff neck, what is the probability of meningitis?"

Key Concept

Bayes' Rule bridges causal and diagnostic reasoning:

$$\underbrace{P(\text{cause} \mid \text{effect})}_{\text{Diagnostic}} = \frac{\overbrace{P(\text{effect} \mid \text{cause})}^{\text{Causal}} \cdot P(\text{cause})}{P(\text{effect})} \tag{12.7}$$

Doctors naturally know $P(\text{symptom} \mid \text{disease})$ (causal), but need $P(\text{disease} \mid \text{symptom})$ (diagnostic) for patient care.

12.4.1 Combining Evidence

When multiple evidence variables are available:

$$P(Y \mid e_1, e_2, \ldots, e_n) = \alpha \cdot P(Y) \prod_{i=1}^{n} P(e_i \mid Y) \tag{12.8}$$

(under the conditional independence assumption).

12.5 Utility Theory

Definition

Utility is a measure of the value or satisfaction that an individual derives from a particular outcome. It is a **personal preference** — different individuals may assign different utilities to the same outcome.

Example (from lecture): A person exchanges a 100-rupee note for a pen. The transaction occurs because the person's utility for the pen exceeds their utility for the 100 rupees. Someone else might make a different choice.

12.5.1 Expected Utility

Expected Utility of an action a:

$$EU(a) = \sum_{s} P(s \mid a) \cdot U(s) \tag{12.9}$$

where $P(s \mid a)$ is the probability of outcome s given action a, and $U(s)$ is the utility of outcome s.

12.5.2 Maximum Expected Utility (MEU) Principle

Definition

A **rational agent** chooses the action that **maximizes expected utility**:

$$a^* = \arg\max_{a} \sum_{s} P(s \mid a) \cdot U(s) \tag{12.10}$$

Example (from lecture): An automated car must choose between two routes:

- Route A: 90% chance of arriving in 20 min, 10% chance of getting stuck for 60 min.
- Route B: 70% chance of arriving in 15 min, 30% chance of a 45-min detour.

The car computes EU(Route A) and EU(Route B) and selects the route with the higher expected utility.

12.5.3 Bayesian Decision Theory

Key Concept

Bayesian Decision Theory = Probability Theory + Utility Theory. It provides a complete framework for rational decision-making under uncertainty:

1. Model uncertainty using probability distributions.
2. Define preferences using utility functions.
3. Choose actions that maximize expected utility.

12.6 Evolution of AI Systems

	Pre-2005	**Post-2005**
Approach	Logic-based systems, probabilistic systems, expert systems, search-based systems	Deep learning, neural networks
Data	Small, curated datasets	Internet-scale datasets
Limitation	Fragile, narrow domains	Need massive compute and data
Key Shift		Non-linear performance scaling with data — more data ⇒ significantly better performance

Harith's Insight / Class Context

From the instructor: Expert systems (pre-2005) were rule-based systems hand-crafted by domain experts. They were brittle: if a situation fell outside the encoded rules, they would fail. The availability of massive datasets post-2005 enabled data-driven approaches (deep learning) that *learn* patterns rather than relying on hand-crafted rules.

12.7 Practice Questions with Solutions

1. **Define conditional independence. How does it simplify joint probability distributions?**

   Solution: X and Y are **conditionally independent** given Z if $P(X, Y \mid Z) = P(X \mid Z) \cdot P(Y \mid Z)$. This simplifies joint distributions by **factoring** them into smaller pieces. Without conditional independence, modeling n variables with k values each requires k^n entries. With conditional independence (e.g., each variable depends on at most m others), complexity reduces to approximately $n \cdot k^m$, which is tractable.

2. **Given $A \perp B \mid C$, express $P(A, B, C)$ in simplified form.**

Solution:

$$\begin{aligned} P(A, B, C) &= P(A, B \mid C) \cdot P(C) \\ &= P(A \mid C) \cdot P(B \mid C) \cdot P(C) \quad \text{(by conditional independence)} \end{aligned}$$

$$\boxed{P(A, B, C) = P(A \mid C) \cdot P(B \mid C) \cdot P(C)}$$

This requires estimating $P(A \mid C)$, $P(B \mid C)$, and $P(C)$ separately, rather than the full joint $P(A, B, C)$.

3. **Explain the difference between causal and diagnostic knowledge. How does Bayes' Rule connect them?**

   Solution: **Causal knowledge** flows cause → effect: $P(\text{symptom} \mid \text{disease})$. This is naturally available (e.g., medical textbooks document that meningitis causes stiff neck). **Diagnostic knowledge** flows effect → cause: $P(\text{disease} \mid \text{symptom})$. This is what we actually need (given a patient's symptoms, what disease do they have?). **Bayes' Rule** bridges them:

$$P(\text{disease} \mid \text{symptom}) = \frac{P(\text{symptom} \mid \text{disease}) \cdot P(\text{disease})}{P(\text{symptom})}$$

   It converts readily available causal knowledge into the needed diagnostic knowledge.

4. **Which route should a rational agent choose?**

   Solution: Compute expected utility for each route:

$$EU(\text{Route 1}) = 0.8 \times 100 + 0.2 \times (-50) = 80 - 10 = \boxed{70}$$
$$EU(\text{Route 2}) = 0.95 \times 80 + 0.05 \times (-20) = 76 - 1 = \boxed{75}$$

   Since $EU(\text{Route 2}) = 75 > EU(\text{Route 1}) = 70$, the rational agent should choose **Route 2**. Note: Route 2 has lower peak utility but higher expected utility due to its greater reliability.

5. **What is the Maximum Expected Utility principle?**

   Solution: The **MEU principle** states that a rational agent should choose the action a^* that maximizes expected utility: $a^* = \arg\max_a \sum_s P(s \mid a) \cdot U(s)$. It is the foundation of rational decision-making because: (1) it systematically accounts for both *uncertainty* (probabilities) and *preferences* (utilities); (2) it provably leads to optimal decisions under the axioms of utility theory; (3) it provides a normative framework — any deviation can be shown to lead to suboptimal outcomes.

6. **Explain the Bayesian approach to combining multiple pieces of evidence under conditional independence.**

   Solution: Given class Y and evidence $e_1, e_2, \ldots, e_n$ that are conditionally independent given Y:

$$P(Y \mid e_1, \ldots, e_n) = \alpha \cdot P(Y) \prod_{i=1}^{n} P(e_i \mid Y)$$

   where $\alpha = 1/P(e_1, \ldots, e_n)$ is a normalizing constant. Each piece of evidence contributes independently to the posterior via its likelihood $P(e_i \mid Y)$. This allows us to **incrementally update** our belief as new evidence arrives, and avoids estimating the intractable full joint $P(e_1, \ldots, e_n \mid Y)$.

7. **Compare the AI landscape pre-2005 and post-2005. What drove the shift to deep learning?**

   Solution: **Pre-2005:** AI was dominated by logic-based systems, expert systems, and probabilistic models. These were hand-crafted, brittle, and limited to narrow domains. They required explicit knowledge engineering. **Post-2005:** Deep learning emerged, powered by (1) **internet-scale datasets** (billions of images, texts); (2) **GPU computing** enabling massive parallel computation; (3) **algorithmic advances** (backpropagation, ReLU, dropout, batch normalization). Key insight: neural networks exhibit **non-linear scaling** — performance improves dramatically with more data, unlike classical methods that plateau.

8. **What is utility? Why is it described as a "personal preference"?**

   Solution: **Utility** is a numerical measure of the value or satisfaction an agent derives from a particular outcome. It is a "personal preference" because different agents assign different utilities to the same outcome. *Example:* A pen worth 100 rupees has high utility to a student who needs it for an exam, but low utility to someone who already owns many pens. Utility captures subjective value, not just objective monetary worth. This is why rational decision-making considers an agent's *own* utility function.

9. **Compute $P(\textbf{disease} \mid \textbf{symptom})$ using Bayes' Rule.**

   Solution: Given: $P(\text{symptom} \mid \text{disease}) = 0.9$, $P(\text{disease}) = 0.01$, $P(\text{symptom}) = 0.05$.

$$\begin{aligned} P(\text{disease} \mid \text{symptom}) &= \frac{P(\text{symptom} \mid \text{disease}) \cdot P(\text{disease})}{P(\text{symptom})} \\ &= \frac{0.9 \times 0.01}{0.05} = \frac{0.009}{0.05} = \boxed{0.18 = 18\%} \end{aligned}$$

   Despite the symptom being highly indicative of the disease ($P = 0.9$), the posterior is only 18% because the disease itself is rare ($P = 0.01$). This illustrates the **base rate fallacy**: ignoring prior probabilities leads to overestimating the probability of rare events.

Part II: Post-Midsem Topics (March–April 2026)

Chapters 13–19 cover the **post-midsem** syllabus (March–April 2026): decision trees (ID3 revisited with worked examples), clustering, decision theory, Markov Decision Processes (value iteration, policy iteration, LP formulation), multi-armed bandits, and state-space search. Each chapter includes fully-worked numerical examples and complete Q&A.

Chapter 13

Decision Trees: ID3 and Information Gain

13.1 Core Concepts

Decision trees recursively partition the input space using axis-aligned splits. The **ID3** algorithm greedily selects the attribute that maximises *information gain*.

13.1.1 Entropy and Information Gain

Formula Reference

$$H(S) = -\sum_{k=1}^{K} p_k \log_2 p_k \qquad \text{(convention: } 0 \log_2 0 = 0\text{)}$$

$$\mathrm{IG}(S, A) = H(S) - \sum_{v \in \mathrm{vals}(A)} \frac{|S_v|}{|S|} H(S_v)$$

ID3 selects $A^* = \arg\max_A \mathrm{IG}(S, A)$ at each split node.

Key Concept

Gain Ratio (C4.5): $\mathrm{GainRatio}(S, A) = \mathrm{IG}(S, A)/\mathrm{SplitInfo}(S, A)$ penalises attributes with many values. **Gini Index** (CART): $\mathrm{Gini}(S) = 1 - \sum_k p_k^2$.

13.1.2 ID3 Algorithm

ID3

ID3$(S, \text{Attributes})$:

1. If all $x \in S$ share the same label ℓ, return leaf(ℓ).
2. If Attributes $= \emptyset$, return leaf(majority label of S).
3. $A^* \leftarrow \arg\max_{A \in \text{Attributes}} \mathrm{IG}(S, A)$.

4. For each value v of A^*: recurse on $(S_v, \text{Attributes} \setminus \{A^*\})$.

13.1.3 Overfitting and Pruning

- **Pre-pruning**: stop splitting when IG falls below threshold (horizon effect risk).
- **Post-pruning**: grow full tree, collapse subtrees that don't improve validation accuracy. Generally preferred.

Exam Tip

(1) Entropy uses $\log_2$ (bits); cross-entropy loss uses ln. (2) Unique-ID attribute achieves maximum IG — use Gain Ratio. (3) IG can be 0 even for a useful attribute if it splits classes symmetrically.

13.2 Worked Example: Tennis Dataset

Decision Tree: Tennis Dataset — Entropy and Information Gain

Setup. 14 examples: 9 Positive (play), 5 Negative. Attributes: Outlook $\in$ {Sunny, Overcast, Rain}, Humidity $\in$ {High, Normal}.
Step 1: Root entropy.

$$H(S) = -\tfrac{9}{14}\log_2\tfrac{9}{14} - \tfrac{5}{14}\log_2\tfrac{5}{14} = \boxed{0.940} \text{ bits}$$

Step 2: IG(Outlook). Sunny: 2+/3−; Overcast: 4+/0−; Rain: 3+/2−.

$$H(S_{\text{Sunny}}) = 0.971, \quad H(S_{\text{Overcast}}) = 0, \quad H(S_{\text{Rain}}) = 0.971$$
$$\text{IG(Outlook)} = 0.940 - [\tfrac{5}{14}(0.971) + \tfrac{4}{14}(0) + \tfrac{5}{14}(0.971)] = \boxed{0.247} \text{ bits}$$

Step 3: IG(Humidity). High: 3+/4−; Normal: 6+/1−.

$$H(S_{\text{High}}) = 0.985, \quad H(S_{\text{Normal}}) = 0.592$$
$$\text{IG(Humidity)} = 0.940 - [\tfrac{7}{14}(0.985) + \tfrac{7}{14}(0.592)] = \boxed{0.151} \text{ bits}$$

Decision. $0.247 > 0.151$: ID3 splits first on **Outlook**. Overcast branch is immediately pure (Play = Yes).

13.3 Practice & Review

Q1. **(MC)** A node has 5 class-A and 5 class-B examples. Entropy =
(A) 0 (B) 0.5 (C) 1 bit (D) 2 bits

Answer: (C). $H = -0.5\log_2 0.5 - 0.5\log_2 0.5 = 1$ bit. Binary entropy peaks at 1 bit for a 50/50 split.

Q2. **(SA)** Why does ID3 prefer high-cardinality attributes, and how does Gain Ratio fix this?

Answer. An attribute with n values partitions data into up to n (possibly pure) subsets, yielding artificially high IG. Gain Ratio divides IG by SplitInfo (entropy of the value distribution), penalising fragmentation and making the criterion scale-invariant.

Q3. **(MC)** IG $= 0$ for all remaining attributes but the node is impure — ID3 should:
(A) Split randomly (B) Return majority-label leaf (C) Error (D) Loop

Answer: (B). Return a leaf with the majority class — the correct base case when no attribute can reduce impurity further.

Q4. **(SA)** Why is post-pruning preferred over pre-pruning?

Answer. Post-pruning evaluates actual overfitting evidence on a held-out set, whereas pre-pruning uses a threshold guess and risks the *horizon effect*: halting before splits that are weak alone but collectively build discriminative structure.

Q5. **(E)** 100 examples (40A, 30B, 30C). X=1: 40A/0B/0C; X=2: 0A/30B/30C. Find IG(X).

Answer. $H(S) = 1.571$ bits; $H(S_{X=1}) = 0$; $H(S_{X=2}) = 1$ bit. IG(X) $= 1.571 - [0.4(0) + 0.6(1)] = \mathbf{0.971}$ bits.

Chapter 14

Clustering and K-Nearest Neighbours: Worked Examples

14.1 K-Means Clustering

K-Means (Lloyd's Algorithm)

Input: points $\{x_1, \ldots, x_n\} \subset \mathbb{R}^d$, k.

1. **Initialise**: choose k centres $\mu_1, \ldots, \mu_k$.
2. **Assignment**: $c_i \leftarrow \arg\min_j \|x_i - \mu_j\|^2$.
3. **Update**: $\mu_j \leftarrow \frac{1}{|C_j|} \sum_{i:c_i=j} x_i$.
4. Repeat 2–3 until convergence.

WCSS and Validation

$J = \sum_j \sum_{i:c_i=j} \|x_i - \mu_j\|^2 \geq 0$; decreases each step $\Rightarrow$ converges (local min).
Silhouette: $s(i) = \frac{b(i)-a(i)}{\max\{a(i),b(i)\}} \in [-1, 1]$. $a(i)$ = intra-cluster mean dist; $b(i)$ = nearest other cluster mean dist.
Davies-Bouldin: $\text{DB} = \frac{1}{k} \sum_i \max_{j \neq i} \frac{\sigma_i + \sigma_j}{d(\mu_i, \mu_j)}$ (lower is better).

K-Means++ initialisation: choose each successive centre w.p. proportional to squared distance from nearest existing centre. Expected WCSS within $O(\log k)$ of optimal.

Exam Tip

K-Means requires k in advance; sensitive to outliers; assumes spherical clusters. Use **elbow method** (plot WCSS vs. k) or silhouette scores to choose k.

14.2 Worked Example: K-Means Iteration Trace

K-Means: 4-Point Example, $k = 2$

Points: $P_1 = (1,1)$, $P_2 = (1,3)$, $P_3 = (5,1)$, $P_4 = (5,3)$. Initial centres: $\mu_1 = (1,1)$, $\mu_2 = (5,3)$.
Iteration 1 — Assignment.

Point	$d(\cdot, \mu_1)$	$d(\cdot, \mu_2)$	**Assign**
P_1	0	$\sqrt{20} \approx 4.47$	C_1
P_2	2	4	C_1
P_3	4	2	C_2
P_4	$\sqrt{20} \approx 4.47$	0	C_2

Update: $\mu_1 \leftarrow (1,2)$; $\mu_2 \leftarrow (5,2)$.
Iteration 2: all assignments unchanged $\Rightarrow$ **Converged.**
Silhouette for $P_1 = (1,1)$: $a(P_1) = d(P_1, P_2) = 2$; $b(P_1) = \frac{1}{2}[4 + \sqrt{20}] = 4.24$. $s(P_1) = \frac{4.24-2}{4.24} \approx \boxed{0.528}$.

14.3 Practice & Review

Q1. **(MC)** K-Means convergence is guaranteed because:

(A) WCSS ≥ 0 and monotonically decreasing (B) Finite number of partitions
(C) Both (D) Centres reach global optimum

Answer: (C). WCSS is bounded below and non-increasing; the number of distinct partitions is finite. Together these preclude cycling. (D) is false: K-Means converges to a local minimum.

Q2. **(SA)** Why is K-Means++ preferred over random initialisation?

Answer. Random init can cluster all centres in one region. K-Means++ selects centres with probability proportional to squared distance from the nearest existing centre, spreading them out and giving expected WCSS within $O(\log k)$ of optimal.

Q3. **(SA)** Interpret $s(i) = -0.3$ for a clustered point.

Answer. $s(i) < 0$ means the point is on average closer to a different cluster than its own — a sign of possible misassignment. Not necessarily an outlier; outliers tend to have $s \approx 0$.

Q4. **(E)** WCSS runs give 450 and 380 for $k = 3$. Which to use? How to choose k?

Answer. Use **380** (lower WCSS). Choose k via the *elbow method* (sharp slowdown in WCSS vs. k) or maximising average silhouette score.

Chapter 15

Decision Theory: MEU and Bayesian Networks (Worked Examples)

15.1 Maximum Expected Utility

MEU Principle

$a^* = \arg\max_a \mathbb{E}[U(o) \mid a] = \arg\max_a \sum_o P(o \mid a)\, U(o)$. Utility U exists (unique up to positive affine transform) iff preferences satisfy vNM axioms. Risk attitude: concave U = risk-averse; linear = risk-neutral; convex = risk-seeking.

Value of Perfect Information

$\text{VPI}(E) = \mathbb{E}_E[\max_a \mathbb{E}[U \mid a, E]] - \max_a \mathbb{E}[U \mid a] \geq 0$. Information never hurts a rational agent.

15.2 D-Separation

Three Canonical BN Patterns

Pattern	Structure	Key independence
Chain	$A \to B \to C$	$A \perp C \mid B$
Fork	$A \leftarrow B \to C$	$A \perp C \mid B$
Collider	$A \to B \leftarrow C$	$A \perp C$, but $A \not\perp C \mid B$

Explaining away: observing a collider (or descendant) activates the path. E.g. Burglary→Alarm←Earthquake: given Alarm=True, Earthquake reduces P(Burglary).

15.3 Worked Example: MEU with Risk-Averse Utility

Decision Theory: Investment Under Risk Aversion

a_1: profit Rs. 100 w.p. 0.6, loss Rs. 50 w.p. 0.4. a_2: certain Rs. 30. $U(x) = \sqrt{x + 100}$.
$U(100) = 14.14$, $U(-50) = 7.07$, $U(30) = 11.40$. $\mathbb{E}[U \mid a_1] = 0.6(14.14) + 0.4(7.07) = 11.31$; $\mathbb{E}[U \mid a_2] = 11.40$. MEU chooses $\mathbf{a_2}$.
Note: $\text{EMV}(a_1) = \text{Rs. } 40 > \text{Rs. } 30$, so a risk-neutral agent prefers a_1. The concave U makes the certain option preferable — risk aversion.

15.4 Practice & Review

Q1. **(MC)** VPI is always:

(A) Positive (B) Negative (C) Non-negative (D) Zero

Answer: (C). $\text{VPI}(E) \geq 0$: a rational agent can ignore new information, so utility cannot decrease. VPI $= 0$ when E would not change the optimal action.

Q2. **(SA)** Explain "explaining away" with an example.

Answer. Burglary→Alarm←Earthquake. Marginally Burglary⊥Earthquake. Observing Alarm=True activates the collider: learning Earthquake occurred reduces $P(\text{Burglary})$, because the earthquake already explains the alarm.

Q3. **(E)** a_1: Rs. 1000 w.p. 0.5, Rs. 0 otherwise. a_2: Rs. 400 certain. $U(x) = \ln(x + 1)$. MEU?

Answer. $\mathbb{E}[U \mid a_1] = 0.5 \ln 1001 = 3.454$; $\mathbb{E}[U \mid a_2] = \ln 401 \approx 5.994$. MEU chooses $\mathbf{a_2}$ (risk-averse; EMV favours a_1 at Rs. 500).

Chapter 16

Markov Decision Processes: Value and Policy Iteration

16.1 MDP Formalism

MDP Definition

Tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$: states, actions, transitions $T(s,a,s') = P(s' \mid s,a)$, rewards $R(s,a)$, discount $\gamma \in [0,1)$. **Markov property**: $P(s_{t+1} \mid s_t, a_t, \ldots) = P(s_{t+1} \mid s_t, a_t)$. Value function: $V^\pi(s) = \mathbb{E}^\pi[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s]$.

16.2 Bellman Equations

Bellman Expectation and Optimality

Policy π: $V^\pi(s) = R(s,\pi(s)) + \gamma \sum_{s'} T(s,\pi(s),s')\, V^\pi(s')$
Optimal: $V^*(s) = \max_a \big[R(s,a) + \gamma \sum_{s'} T(s,a,s')\, V^*(s')\big]$
$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} T(s,a,s') V^*(s')$; $\quad \pi^*(s) = \arg\max_a Q^*(s,a)$.

16.3 Value Iteration

Value Iteration

Init $V_0 = 0$. Repeat: $V_{k+1}(s) \leftarrow \max_a [R(s,a) + \gamma \sum_{s'} T(s,a,s') V_k(s')]$. Stop when $\|V_{k+1} - V_k\|_\infty < \varepsilon(1-\gamma)/(2\gamma)$. Contraction: $\|V_k - V^*\|_\infty \le \gamma^k \|V_1 - V_0\|_\infty / (1-\gamma)$.

16.4 Policy Iteration

Policy Iteration

1. **Eval**: solve V^{π_k} via linear system ($O(|\mathcal{S}|^3)$).
2. **Improve**: $\pi_{k+1}(s) = \arg\max_a [R(s,a) + \gamma \sum_{s'} T(s,a,s') V^{\pi_k}(s')]$.
3. Stop when $\pi_{k+1} = \pi_k$.

Policy iteration uses fewer steps but each step is costlier than value iteration.

16.5 Worked Example: Grid World Value Iteration

MDP: Grid World 4×3 — Value Iteration Trace

4×3 grid; $(4,3) = +1$, $(4,2) = -1$ (terminals); wall at $(2,2)$; step cost -0.04; $\gamma = 1$. Transitions: 0.8 intended, 0.1 each perpendicular.
$V_0 = 0$ everywhere. **After 1 step**: $V_1(4,3) = 1$; $V_1(3,3) = -0.04 + 0.8(1) = \mathbf{0.76}$; others $= -0.04$.
After 2 steps: $V_2(3,3) \approx 0.912$; $V_2(2,3) \approx 0.568$; $V_2(3,2) \approx 0.464$.
Converged V^*:

Row\Col	1	2	3	4
3	0.812	0.868	0.918	**+1.0**
2	0.762	*wall*	0.660	**−1.0**
1	0.705	0.655	0.611	0.388

Optimal policy: go Right/Up toward $(4,3)$; at $(3,1)$ go Up (not Right) to avoid slipping into -1.

16.6 Practice & Review

Q1. **(MC)** The Markov property states that:

(A) Future depends only on current state and action (B) Future is independent of all past
(C) Policy must be stochastic (D) Rewards must be bounded

Answer: (A). $P(s_{t+1} \mid s_t, a_t, \ldots) = P(s_{t+1} \mid s_t, a_t)$. (B) is too strong: future depends on the current state, not the full history.

Q2. **(SA)** Why is $\gamma < 1$ needed for infinite-horizon MDPs?

Answer. Without discounting, $\sum_{t=0}^{\infty} R_t$ may diverge. $\gamma < 1$ ensures $|V^\pi(s)| \leq R_{\max}/(1-\gamma) < \infty$. It also models the time value of money and agent survival probability.

Q3. **(SA)** Prove the Bellman operator is a contraction with factor γ.

Answer. For any V, W: $|(TV)(s) - (TW)(s)| \leq \gamma \max_a \sum_{s'} T(s,a,s')|V(s') - W(s')| \leq \gamma \|V - W\|_\infty$. Taking max over s: $\|TV - TW\|_\infty \leq \gamma \|V - W\|_\infty$. Since $\gamma < 1$, Banach's theorem guarantees a unique fixed point V^*.

Q4. **(E)** 2-state MDP: $R(s_1, a_1) = 5$, $P(s_2 \mid s_1, a_1) = 0.3$; $R(s_1, a_2) = 2$, $P(s_2 \mid s_1, a_2) = 0.7$; $R(s_2) = 10$, $P(s_2 \mid s_2) = 1$. $\gamma = 0.9$. Find $V^*(s_1)$, $V^*(s_2)$.

Answer. $V^*(s_2) = 10 + 0.9V^*(s_2) \Rightarrow V^*(s_2) = 100$. $Q(s_1, a_1) = 5 + 0.9[30 + 0.7V^*] = 32 + 0.63V^*$; $Q(s_1, a_2) = 2 + 0.9[70 + 0.3V^*] = 65 + 0.27V^*$. Test a_2: $V^* = 65 + 0.27V^* \Rightarrow V^*(s_1) \approx 89.04$. Check $Q(s_1, a_1) = 88.09 < 89.04$. ✓ Optimal: a_2.

Chapter 17

Markov Decision Processes: Linear Programming Formulation

17.1 Primal LP

Primal LP

V^* is the *smallest* function satisfying all Bellman constraints:

$$\min_V \sum_s \alpha(s)\, V(s) \quad \text{s.t.} \quad V(s) \geq R(s,a) + \gamma \sum_{s'} T(s,a,s') V(s') \quad \forall s, a$$

Variables: $|\mathcal{S}|$. Constraints: $|\mathcal{S}||\mathcal{A}|$.

17.2 Dual LP: Occupation Measure

Dual LP

$x(s,a) \geq 0$ = discounted expected visits to (s,a) under optimal policy.

$$\max_x \sum_{s,a} R(s,a)\, x(s,a) \quad \text{s.t.} \quad \sum_a x(s',a) = \alpha(s') + \gamma \sum_{s,a} T(s,a,s') x(s,a) \quad \forall s'$$

Optimal policy: $\pi^*(s) = \arg\max_a x^*(s,a)$.

LP extends naturally to **Constrained MDPs**: add linear constraints on $x(s,a)$ (e.g. safety, budget). This is the main advantage over value/policy iteration.

17.3 Worked Example: LP for a 2-State MDP

MDP-LP: 2-State, 2-Action

States $\{s_1, s_2\}$; s_1 actions $\{a_1, a_2\}$; s_2 action noop. $R(s_1, a_1) = 2$, $R(s_1, a_2) = 4$, $R(s_2) = 0$. $T(s_1, a_1, s_2) = 0.5$, $T(s_1, a_2, s_2) = 0.8$; $T(s_2, s_2) = 1$. $\gamma = 0.9$.
Variables: $x_{11} = x(s_1, a_1)$, $x_{12} = x(s_1, a_2)$. **Objective**: $\max 2x_{11} + 4x_{12}$.
Flow for s_1: $x_{11} + x_{12} = 1 + 0.9[0.5x_{11} + 0.2x_{12}] \Rightarrow 0.55x_{11} + 0.82x_{12} = 1$.
Maximise $2x_{11} + 4x_{12}$ subject to $0.55x_{11} + 0.82x_{12} = 1$, $x \geq 0$. Reward-per-constraint-unit: $4/0.82 \approx 4.88 > 2/0.55 \approx 3.64$. Set $x_{11} = 0$, $x_{12} \approx 1.220$. Optimal policy: $\pi^*(s_1) = \mathbf{a_2}$.

17.4 Practice & Review

Q1. **(MC)** Primal LP variables count:
(A) $|\mathcal{S}||\mathcal{A}|$ (B) $|\mathcal{S}|$ (C) $|\mathcal{A}|$ (D) $|\mathcal{S}|^2$

Answer: (B). One variable $V(s)$ per state; $|\mathcal{S}||\mathcal{A}|$ constraints (Bellman inequalities).

Q2. **(SA)** What does $x(s, a)$ represent and how is π^* extracted?

Answer. $x(s, a)$ is the discounted expected number of times the agent takes action a in state s. The optimal policy is $\pi^*(s) = \arg\max_a x^*(s, a)$.

Q3. **(SA)** Why is the LP formulation useful even when VI is available?

Answer. (i) Mature LP solvers may outperform hand-coded VI. (ii) Constrained MDPs (safety/budget) extend naturally by adding constraints on $x(s, a)$. (iii) LP duality gives sensitivity analysis for free.

Chapter 18

Multi-Armed Bandits

18.1 Problem Setup and Regret

K arms; at round t pull arm a_t, receive $r_t \sim P_{a_t}$ with mean μ_{a_t}.

Regret

$\text{Regret}(T) = T\mu^* - \sum_{t=1}^{T} \mu_{a_t}$, $\mu^* = \max_k \mu_k$. Instance-dependent: $R(T) = \sum_{k:\mu_k<\mu^*} \Delta_k \mathbb{E}[N_k(T)]$, $\Delta_k = \mu^* - \mu_k$.

18.2 Algorithms

ε-Greedy

With prob ε: random arm (explore). With prob $1-\varepsilon$: $\arg\max_k \hat{\mu}_k$ (exploit). Fixed ε: $R(T) = O(\varepsilon T)$; decaying $\varepsilon_t = c/t$: $R(T) = O(\log T)$.

UCB1

Pull each arm once; then pull $a_t = \arg\max_k [\hat{\mu}_k + \sqrt{2 \ln t / N_k(t-1)}]$. Regret bound: $R(T) \le \sum_{k:\Delta_k>0} \frac{8 \ln T}{\Delta_k} + O(K)$ — optimal $O(\log T)$.

Thompson Sampling

Beta$(1,1)$ prior for each arm; at each round sample $\tilde{\mu}_k \sim \text{Beta}(\alpha_k, \beta_k)$; pull arm with highest sample; update α (success) or β (failure). Achieves $O(\log T)$ regret; empirically often outperforms UCB1.

Lai–Robbins Lower Bound

$\liminf_{T\to\infty} R(T)/\ln T \ge \sum_{k:\Delta_k>0} \Delta_k / \text{KL}(\mu_k \| \mu^*)$. No consistent algorithm can achieve $o(\log T)$ regret. UCB1 and Thompson Sampling are asymptotically optimal.

Exam Tip

Instance-dependent bound: $O(\sum_k \ln T/\Delta_k)$. Instance-independent (minimax): $O(\sqrt{KT\ln K})$. UCB1 achieves both.

18.3 Worked Example: UCB1 8-Round Trace

UCB1: 3 Arms, 8 Rounds

Arms with $\mu_1 = 0.9$, $\mu_2 = 0.7$, $\mu_3 = 0.5$ (deterministic). Init: pull each once (R1–R3).
Round 4 ($\ln 4 = 1.386$, bonus $= \sqrt{2(1.386)} = 1.665$): $\text{UCB}_1 = 2.565$, $\text{UCB}_2 = 2.365$, $\text{UCB}_3 = 2.165$. Pull **A1**.
Round 5 ($\ln 5 = 1.609$): $\text{UCB}_1 = 0.9 + \sqrt{3.22/2} = 2.17$, $\text{UCB}_2 = 0.7 + \sqrt{3.22} = 2.49$, $\text{UCB}_3 = 2.29$. Pull **A2**.
Round 6 ($\ln 6 = 1.792$): $\text{UCB}_1 = 2.24$, $\text{UCB}_2 = 2.04$, $\text{UCB}_3 = 0.5 + \sqrt{3.58} = 2.39$. Pull **A3**.
Round 7: $\text{UCB}_1 = 2.29 > \text{UCB}_2 = 2.09 > \text{UCB}_3 = 1.89$. Pull **A1**.
Round 8: $\text{UCB}_2 = 2.14 > \text{UCB}_1 = 2.08 > \text{UCB}_3 = 1.94$. Pull **A2**.
Pulls: A1×3, A2×3, A3×2. UCB naturally concentrates on better arms as bonuses shrink.

18.4 Practice & Review

Q1. **(MC)** UCB1 exploration bonus is proportional to:
(A) $\sqrt{\ln t/N_k}$ (B) $1/N_k$ (C) ε (D) Δ_k

Answer: (A). $\sqrt{2\ln t/N_k}$ grows logarithmically with t (ensures underexplored arms are tried) and shrinks with N_k (reduces exploration as data accumulates). Achieves $O(\log T)$ regret.

Q2. **(SA)** How does UCB1 balance exploration and exploitation?

Answer. UCB1 adds an optimistic confidence bonus to each empirical mean. Arms not recently pulled have large $\ln t/N_k$ (high bonus $\Rightarrow$ more exploration). As arms are pulled more, the bonus shrinks and the index converges to the true mean (exploitation). No ε schedule needed.

Q3. **(MC)** After Beta(1,1) prior and 3 successes + 2 failures, the posterior is:
(A) Beta(3,2) (B) Beta(4,3) (C) Beta(5,3) (D) Beta(3,3)

Answer: (B). Update: $\alpha = 1+3 = 4$, $\beta = 1+2 = 3$. Posterior = Beta(4,3).

Q4. **(E)** $\mu_1 = 0.9$, $\mu_2 = 0.6$, $\mu_3 = 0.3$; after 100 rounds A1 pulled 70, A2 20, A3 10 times. Compute regret.

Answer. $\Delta_2 = 0.3$, $\Delta_3 = 0.6$. $R(100) = 0(70) + 0.3(20) + 0.6(10) = 0 + 6 + 6 = \mathbf{12}$. Total possible: 90. Collected: $70(0.9) + 20(0.6) + 10(0.3) = 78$. Regret $= 90 - 78 = 12$. ✓

Chapter 19

State-Space Search

19.1 Problem Formulation and Properties

Search Properties

Complete	Finds a solution if one exists
Optimal	Finds minimum-cost solution
Time	Nodes expanded (fn of b, d, cost)
Space	Max nodes in memory

19.2 Uninformed Search

Comparison Table

Algorithm	Complete	Optimal	Time	Space
BFS	Yes	Yes (unit)	$O(b^d)$	$O(b^d)$
DFS	No	No	$O(b^m)$	$O(bm)$
IDDFS	Yes	Yes (unit)	$O(b^d)$	$O(bd)$
UCS	Yes	Yes ($c \geq 0$)	$O(b^{C^*/\varepsilon})$	$O(b^{C^*/\varepsilon})$

IDDFS = BFS space $O(bd)$ + BFS completeness/optimality. Overhead factor $\leq b/(b-1)$.

19.3 A* Search

A* Search

Priority queue ordered by $f(n) = g(n) + h(n)$: $g(n)$ = cost from start; $h(n)$ = heuristic to goal. **Complete and optimal** when h is *admissible*: $h(n) \leq h^*(n)$. **Consistent** (triangle inequality): $h(n) \leq c(n, a, n') + h(n')$. Consistency $\Rightarrow$ Admissibility. $h = 0$: reduces to UCS. $g = 0$: reduces to Greedy Best-First.

Exam Tip

Admissible heuristics: straight-line distance (navigation), misplaced tiles / Manhattan distance (8-puzzle). Derived by **relaxing problem constraints**. Non-admissible h: faster but possibly suboptimal.

19.4 Worked Example: A* on Romania Map

A*: Romania — Arad to Bucharest

Heuristics h (straight-line to Bucharest): Arad 366, Sibiu 253, Rimnicu 193, Pitesti 100, Fagaras 176. Road costs: A→S: 140; S→R: 80; S→F: 99; R→P: 97; P→B: 101; F→B: 211.
Expansion trace ($f = g+h$): A(366)→S(393)→R(413)→F(415)→P(417)→B via P(418). **GOAL.**
Optimal path: A→S→R→P→B, cost $= 140 + 80 + 97 + 101 =$ **418** km.
Greedy takes A→S→F→B = 450 km (follows h greedily but hits expensive edge F→B).

19.5 Practice & Review

Q1. **(MC)** Primary difference between BFS and DFS:

(A) BFS level-by-level; DFS depth-first (B) BFS uses stack; DFS queue
(C) BFS for unweighted; DFS for weighted (D) BFS finds longest path

Answer: (A). BFS uses a FIFO queue; DFS uses LIFO stack. (B) has the data structures reversed.

Q2. **(SA)** Advantages of IDDFS over BFS and DFS?

Answer. IDDFS has BFS's completeness and optimality (unit costs) with DFS's $O(bd)$ space. The overhead is small: total extra work factor $\leq b/(b-1) \approx 1.13$ for $b = 10$.

Q3. **(MC)** Admissible heuristic means:

(A) $h(n) \leq c(n, a, n') + h(n')$ (B) $h(n) \leq h^*(n)$ (C) $h(n) \geq h^*(n)$ (D) h is exact

Answer: (B). Admissibility: h never overestimates the true cost. (A) is consistency (stronger).

Q4. **(SA)** Explain the difference between admissibility and consistency.

Answer. Admissibility: $h(n) \leq h^*(n)$ (global bound). Consistency: $h(n) \leq c(n, a, n') + h(n')$ (triangle inequality on h). Consistency ⇒ admissibility. Consistency additionally ensures f-values are non-decreasing along paths, so A* expands each node at most once (no re-expansion needed).

Q5. **(E)** Apply A* to: $h(s) = 5$, $h(a) = 3$, $h(b) = 4$, $h(g) = 0$; edges $s \to a$ (2), $s \to b$ (4), $a \to g$ (5), $b \to g$ (2). Is h admissible?

Answer. $h^*(b) = 2$ but $h(b) = 4 > 2$: **not admissible.** A* with this h expands $s \to a \to g$ (cost 7) before b and declares cost-7 optimal, missing $s \to b \to g$ (cost 6). This demonstrates that non-admissible heuristics break A*'s optimality guarantee.

Post-Midsem Formula Reference

Decision Trees

$H(S) = -\sum_k p_k \log_2 p_k$; $\quad \mathrm{IG}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$

Clustering

$J = \sum_j \sum_{i:c_i=j} \|x_i - \mu_j\|^2$; $\quad s(i) = \frac{b(i)-a(i)}{\max\{a(i),b(i)\}}$

MDPs

$V^*(s) = \max_a [R(s,a) + \gamma \sum_{s'} T(s,a,s')V^*(s')]$; $\quad \pi^*(s) = \arg\max_a Q^*(s,a)$

Bandits

$\mathrm{UCB1}_k(t) = \hat{\mu}_k + \sqrt{2 \ln t / N_k}$; $\quad R(T) \leq \sum_k \frac{8 \ln T}{\Delta_k} + O(K)$

A* Search

$f(n) = g(n) + h(n)$; $\quad$ Admissible: $h(n) \leq h^*(n)$; $\quad$ Consistent: $h(n) \leq c(n, a, n') + h(n')$

Key Theorems

1. Bellman operator is a γ-contraction in L^∞; unique fixed point V^*.
2. Policy improvement theorem: π' greedy w.r.t. $V^\pi \Rightarrow V^{\pi'} \geq V^\pi$.
3. Lai–Robbins: no consistent bandit algorithm achieves $o(\log T)$ regret.
4. A* optimal under admissibility (tree search); consistent heuristic $\Rightarrow$ no re-expansion.

Appendix A

Quick Reference: Key Formulas

A.1 Statistics and Error Metrics

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 \tag{A.1}$$

$$\text{MSE Decomposition} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error} \tag{A.2}$$

A.2 Classification Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{A.3}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{A.4}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{A.5}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{A.6}$$

A.3 Probability

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} \quad \text{(Bayes' Rule)} \tag{A.7}$$

$$P(A) = \sum_{b} P(A, B = b) \quad \text{(Marginalization)} \tag{A.8}$$

$$P(A, B \mid C) = P(A \mid C) \cdot P(B \mid C) \quad \text{(Conditional Independence)} \tag{A.9}$$

A.4 Association Rules

$$\text{Support}(X \to Y) = \frac{|X \cup Y|}{|D|} \tag{A.10}$$

$$\text{Confidence}(X \to Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)} \tag{A.11}$$

$$\text{Lift}(X \to Y) = \frac{\text{Confidence}(X \to Y)}{\text{Support}(Y)} \tag{A.12}$$

A.5 Information Theory

$$H(S) = -\sum_{i=1}^{k} p_i \log_2 p_i \quad \text{(Entropy)} \tag{A.13}$$

$$IG(S, A) = H(S) - \sum_{v} \frac{|S_v|}{|S|} H(S_v) \quad \text{(Information Gain)} \tag{A.14}$$

$$\text{Gini}(S) = 1 - \sum_{i=1}^{k} p_i^2 \quad \text{(Gini Index)} \tag{A.15}$$

A.6 Distance Metrics

$$d_{\text{Euclidean}}(x, x') = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2} \tag{A.16}$$

$$d_{\text{Manhattan}}(x, x') = \sum_{j=1}^{d} |x_j - x'_j| \tag{A.17}$$

$$d_{\text{Hamming}}(x, x') = \sum_{j=1}^{d} \mathbb{1}[x_j \neq x'_j] \tag{A.18}$$

A.7 Regularization

$$\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{data}} + \lambda \cdot R(\mathbf{w}) \tag{A.19}$$

$$R_{\text{L1}} = \sum_{j} |w_j| \quad \text{(Lasso)} \tag{A.20}$$

$$R_{\text{L2}} = \sum_{j} w_j^2 \quad \text{(Ridge)} \tag{A.21}$$

A.8 Feature Scaling

$$x'_{\text{standardized}} = \frac{x - \mu}{\sigma} \tag{A.22}$$

$$x'_{\text{normalized}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{A.23}$$

A.9 Clustering

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \quad \text{(K-Means Objective)} \tag{A.24}$$

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad \text{(Silhouette Score)} \tag{A.25}$$

A.10 Decision Theory

$$EU(a) = \sum_{s} P(s \mid a) \cdot U(s) \quad \text{(Expected Utility)} \tag{A.26}$$

$$a^* = \arg\max_{a} EU(a) \quad \text{(MEU Principle)} \tag{A.27}$$

Appendix B

Comprehensive Practice Exam

B.1 Multiple Choice Questions

1. The main goal of a machine learning algorithm is to:

   a) Minimize the training error
   b) Minimize the mean square error on unseen data
   c) Maximize the model complexity
   d) Memorize the training data

   Answer: (b)

2. Which type of learning uses labeled data?

   a) Unsupervised learning
   b) Reinforcement learning
   c) Supervised learning
   d) Semi-supervised learning

   Answer: (c)

3. Overfitting occurs when:

   a) The model is too simple
   b) The model captures noise in the training data
   c) Training and test errors are both high
   d) The model has high bias

   Answer: (b)

4. In k-fold cross-validation, the final performance estimate is:

   a) The best score among the k folds
   b) The worst score among the k folds
   c) The average score across all k folds

d) The score on the last fold only

Answer: (c)

5. The primary challenge in learning the likelihood of a distribution is:

   a) The number of parameters grows exponentially with features
   b) The data is always noisy
   c) Bayes' theorem is hard to compute
   d) There are too many classes

   Answer: (a)

6. The Naïve Bayes assumption states that:

   a) Features are independent
   b) Features are conditionally independent given the class
   c) All features are equally important
   d) Classes are equally probable

   Answer: (b)

7. Lift > 1 for an association rule indicates:

   a) Negative association
   b) No association
   c) Positive association
   d) Random co-occurrence

   Answer: (c)

8. Which of the following is NOT a property of a valid distance metric?

   a) Non-negativity
   b) Asymmetry
   c) Triangle inequality
   d) Identity of indiscernibles

   Answer: (b) — Distance metrics must be *symmetric*, not asymmetric.

9. What does the Gini index measure?

   a) Information gain
   b) Node impurity
   c) Distance between clusters
   d) Prediction accuracy

   Answer: (b)

10. A rational agent in decision theory should:

    a) Maximize the probability of success

    b) Minimize risk

    c) Maximize expected utility

    d) Always choose the safest option

    Answer: (c)

B.2 Short Answer Questions with Solutions

1. **Explain the bias-variance tradeoff.**

   Solution: As model complexity increases: **Bias** decreases (model captures more patterns), **Variance** increases (model becomes more sensitive to training data fluctuations). **Training error** monotonically decreases. **Test error** forms a U-shape: initially decreases (better fit), then increases (overfitting). The optimal complexity is at the minimum of test error, where bias and variance are balanced. Mathematically: $\text{MSE} = \text{Bias}^2 + \text{Variance} + \sigma^2_{\text{noise}}$.

2. **Compute support, confidence, and lift for $A \to B$.**

   Solution: $|D| = 200$, $|A| = 80$, $|B| = 60$, $|A \cap B| = 30$.

$$\text{Support}(A \to B) = \frac{30}{200} = \boxed{0.15 = 15\%}$$

$$\text{Confidence}(A \to B) = \frac{30}{80} = \boxed{0.375 = 37.5\%}$$

$$\text{Lift}(A \to B) = \frac{0.375}{60/200} = \frac{0.375}{0.30} = \boxed{1.25}$$

   Lift > 1 indicates a positive association between A and B.

3. **Compute the entropy for 10 positive, 5 negative examples.**

   Solution: Total $= 15$. $p_+ = 10/15 = 2/3$, $p_- = 5/15 = 1/3$.

$$\begin{aligned} H &= -\frac{2}{3}\log_2\frac{2}{3} - \frac{1}{3}\log_2\frac{1}{3} \\ &= -\frac{2}{3}(-0.585) - \frac{1}{3}(-1.585) \\ &= 0.390 + 0.528 = \boxed{0.918 \text{ bits}} \end{aligned}$$

4. **Why does L1 perform feature selection but L2 does not?**

   Solution: **L1** ($\sum |w_j|$) has a diamond-shaped constraint region in weight space. The optimal solution tends to lie at a **corner** of the diamond, where one or more weights are exactly zero — effectively removing those features. **L2** ($\sum w_j^2$) has a circular constraint region with no corners. The optimal solution shrinks all weights toward zero but *never reaches exactly zero*. Therefore L1 produces **sparse** solutions (feature selection) while L2 produces **dense** solutions (all features retained with small weights).

5. **Describe K-Means. Objective function? Convergence? Global optimum?**

   Solution: **Algorithm:** (1) Initialize K centroids randomly. (2) Assign each point to nearest centroid. (3) Update centroids as cluster means. (4) Repeat until convergence. **Objective:** Minimize within-cluster sum of squares $J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$. **Convergence: Yes** — J decreases at each step (assignment and update both reduce or maintain J), and J is bounded below by 0, so it must converge. **Global optimum: No** — the problem is NP-hard. K-Means finds a local minimum that depends on initialization.

B.3 Long Answer / Essay Questions with Solutions

1. **Classification Pipeline (1000 samples, 50 features, 3 classes).**

   Solution:

   1. **Data cleaning:** Profile data (check types, distributions). Handle missing values (impute with mean/median or drop). Remove duplicates. Identify and handle outliers.
   2. **Feature engineering:** Encode categorical features (one-hot encoding). Scale numerical features (standardization for NB, not strictly needed for DT).
   3. **Feature selection:** With 50 features, use embedded methods (e.g., train a decision tree to get feature importances) or filter methods (mutual information) to select the top 15–20 features.
   4. **Model selection:**
      - **Naïve Bayes:** Fast, handles high dimensions well, assumes feature independence (often violated). Good baseline.
      - **Decision Trees:** Interpretable, handles feature interactions, prone to overfitting. Use pruning or limit depth.
   5. **Cross-validation:** Use **nested 5-fold CV**: outer loop evaluates models, inner loop tunes hyperparameters (e.g., max_depth for DT, smoothing for NB). This prevents data leakage.
   6. **Evaluation:** Since 3 classes, use **macro-averaged F1 score** (handles potential class imbalance), confusion matrix, and per-class precision/recall.
   7. **Pipeline:** Wrap all steps in an sklearn Pipeline to prevent data leakage.

2. **Decision Theory: Self-driving car at intersection.**

   Solution: Define utilities (higher = better):

	No collision	**Collision**
Go straight	$U = 90$ (fast, safe)	$U = -1000$ (catastrophic)
Detour	$U = 70$ (slower, safe)	$U = -1000$ (catastrophic)

   Compute expected utilities:

$$EU(\text{straight}) = 0.95 \times 90 + 0.05 \times (-1000) = 85.5 - 50 = \boxed{35.5}$$

$$EU(\text{detour}) = 0.999 \times 70 + 0.001 \times (-1000) = 69.93 - 1.0 = \boxed{68.93}$$

Recommendation: Choose the **detour** ($EU = 68.93 > 35.5$). The much lower collision risk of the detour outweighs the time cost. This demonstrates how high-consequence low-probability events dominate the expected utility calculation — rational agents are risk-sensitive to catastrophic outcomes.

3. **Conditional Independence: Why it is "one of the most important concepts in AI."**

   Solution: Consider a medical diagnosis system with 5 binary symptoms $S_1, \ldots, S_5$ and a disease variable D.

   Without conditional independence: The full joint $P(S_1, S_2, S_3, S_4, S_5, D)$ requires $2^6 - 1 = 63$ parameters.

   With conditional independence (each symptom is independent of other symptoms given the disease):

   $$P(S_1, \ldots, S_5, D) = P(D) \prod_{i=1}^{5} P(S_i \mid D)$$

   Parameters needed: $P(D) = 1$ parameter, each $P(S_i \mid D) = 2$ parameters (one for $D = T$, one for $D = F$). Total $= 1 + 5 \times 2 = \mathbf{11}$ parameters.

   Reduction: from 63 to 11 parameters — a factor of nearly $6\times$. For larger problems (50 variables with 5 values each), the full joint has $5^{50} \approx 10^{35}$ entries, while conditional independence can reduce this to $50 \times 5^5 \approx 156{,}250$ — making the problem tractable.

   This is why conditional independence is fundamental: it enables probabilistic systems to **scale up** to real-world domains. It underlies Bayesian networks, Naïve Bayes, Hidden Markov Models, and most practical probabilistic AI systems. As noted in class, conditional independence is more common than absolute independence in practice, and decomposing large tables into weakly connected subsets is one of the most important developments in AI.